



Bit-precise integers

Document number:

[P3666R4](#)

Date:

2026-05-12

Audience:

CWG, LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Jan Schultke <janschultke@gmail.com>

GitHub Issue:

wg21.link/P3666/github

Source:

github.com/Eisenwave/cpp-proposals/blob/master/src/bitint.cow



ABSTRACT: C23 has introduced so-called "bit-precise integers" into the language, which should be brought to C++ for compatibility, among other reasons. Following an exploration of possible designs in [\[P3639R0\]](#) "The `_BitInt` Debate", this proposal introduces a new set of fundamental types to C++.

§ Contents

- 1 **Revision history**
 - 1.1 Changes since R3
 - 1.2 Changes since R2
 - 1.3 Changes since R1
 - 1.4 Changes since R0
- 2 **Introduction**
 - 2.1 C23
 - 2.2 P3140R0 "`std::int_least128_t`"
 - 2.3 P3639R0 "The `_BitInt` Debate"
- 3 **Motivation**
 - 3.1 Computation beyond 64 bits
 - 3.2 Cornerstone of standard library facilities



3.3	C ABI compatibility
3.4	Resolving issues with the current integer type system
3.5	Portable exact-width integers
4	Core design
4.1	Why not a class template?
4.1.1	LEWG is not convinced that a library type should be pursued
4.1.2	Full C compatibility requires fundamental types
4.1.3	Common spelling of unsigned <code>_BitInt(N)</code>
4.1.4	C compatibility would require an enormous amount of operator overloads etc.
4.1.5	Constructors cannot signal narrowing
4.1.6	Tiny integers are useful in C++
4.1.7	Special deduction rules
4.1.8	Special overload resolution rankings
4.1.9	Quality of implementation requires a fundamental type
4.2	Why the <code>_BitInt</code> keyword spelling?
4.3	Underlying type of enumerations
4.4	Should bit-precise integers be optional?
4.5	<code>_BitInt(1)</code>
4.6	Undefined behavior on signed integer overflow
4.7	Permissive implicit conversions
4.7.1	C compatibility
4.7.2	Difficulty of carving out exceptions in the language
4.7.3	Picking some low-hanging fruits
4.7.4	Conclusion on implicit conversions
4.8	Raising the <code>BITINT_MAXWIDTH</code>
4.8.1	Possible increased <code>BITINT_MAXWIDTH</code> values
4.9	Template argument deduction
4.10	No preprocessor changes, for better or worse
4.11	Padding in <code>_BitInt</code>
5	Library design
5.1	Preventing library support for <code>_BitInt</code>
5.2	Broadening <code>is_integral</code>
5.2.1	Reasons for making <code>std::is_integral_v<_BitInt(N)></code> true
5.2.2	Conclusion
5.3	<code>make_signed</code> and <code>make_unsigned</code>
5.4	The problem of representing widths as <code>int</code>
5.5	Preventing <code>ranges::iota_view</code> ABI break
5.6	Preserving integer-class types
5.7	Bit-precise <code>size_t</code> , <code>ptrdiff_t</code>
5.8	Feature testing
5.9	Using bit-precise integers in <code><cmath></code> functions
5.10	Note on alias templates for <code>_BitInt</code>



6	Implementation experience
7	Impact on the standard
7.1	Impact on the core language
7.2	Impact on the standard library
8	Wording
8.1	Core
8.1.1	[lex.icon]
8.1.2	[basic.fundamental]
8.1.3	[conv.rank]
8.1.4	[conv.prom]
8.1.5	[dcl.type.general]
8.1.6	[dcl.type.simple]
8.1.7	[dcl.enum]
8.1.8	[temp.deduct.general]
8.1.9	[temp.deduct.type]
8.1.10	[cpp.predefined]
8.1.11	[diff.lex]
8.2	Library
8.2.1	[allocator.requirements.general]
8.2.2	[version.syn]
8.2.3	[support.types.byteops]
8.2.4	[cstdint.syn]
8.2.5	[climits.syn]
8.2.6	[intseq.intseq]
8.2.7	[meta.trans.sign]
8.2.8	[utility.intcmp]
8.2.9	[bit.byteswap]
8.2.10	[bit]
8.2.11	[stdbit.h.syn]
8.2.12	[container.reqmts]
8.2.13	[mdspan.extents.overview]
8.2.14	[mdspan.sub.overview]
8.2.15	[mdspan.sub.range.slices]
8.2.16	[iterator.concept.winc]
8.2.17	[iterator.iterators]
8.2.18	[common.iter.types]
8.2.19	[range.iota.view]
8.2.20	[alg.foreach]
8.2.21	[alg.search]
8.2.22	[alg.copy]
8.2.23	[alg.fill]
8.2.24	[alg.generate]
8.2.25	[numeric.ops.gcd]
8.2.26	[numeric.ops.lcm]



8.2.27	[numeric.ops.midpoint]
8.2.28	[numeric.sat.func]
8.2.29	[numeric.sat.cast]
8.2.30	[charconv.syn]
8.2.31	[format.formatter.spec]
8.2.32	[cmplx.over]
8.2.33	[rand.req.seedseq]
8.2.34	[rand.req.urng]
8.2.35	[rand.util.seedseq]
8.2.36	[cmath.syn]
8.2.37	[simd.expos]
8.2.38	[simd.expos.defn]
8.2.39	[simd.mask.overview]
8.2.40	[simd.mask.ctor]
8.2.41	[numerics.c.ckdint]
8.2.42	[time.duration.general]
8.2.43	[stream.types]
8.2.44	[atomics.ref.int]

9 Acknowledgements

10 References

§ 1. Revision history

§ 1.1. Changes since R3

During the 2026-03 meeting in Croydon, both EWG and LEWG saw the paper. EWG had only the following poll:

”

Forward P3666R3 to CWG and LEWG for C++29

SF	F	N	A	SA
17	29	3	2	0

Result: strong consensus in favor

The following changes were made based on feedback from both groups (where the design changes are all to the library part, since EWG forwarded with no changes):

- mentioned the [auto](#) deduction case in example in [§4.9. Template argument deduction](#) (no design change, just more discussion that matches the wording)
- added discussion on [§4.11. Padding in _BitInt](#)
- included LEWG poll results from the 2026-03 Croydon meeting and called for relitigation in [§5.2. Broadening is_integral](#)



- removed the proposed `std::bit_int` and `std::bit_uint` alias templates
- removed all proposed library overloads for `_BitInt`, reducing library support to the bare minimum
- removed the section on Education and Teaching principles; it no longer added value because `_BitInt` is obviously a C compatibility feature, and this is evident from other sections of the paper (and the keyword) already
- mentioned that [N3705] has been adopted into C2y and rephrased parts of the paper accordingly
- rebased §8. Wording on [N5032] with the changes from the Croydon motions applied
- fixed various editorial problems
- massively overhauled wording in §8.2. Library to prevent `_BitInt` support

§ 1.2. Changes since R2

- mentioned that [N3747] has been approved by WG14 for C2y
- removed `std::simd` support
- removed `std::atomic` support
- removed mentions of P3438R0 because `to_string` is now `constexpr`
- removed changes to [utility.intcmp] because these are no longer necessary after editorial pull request #8616 was merged
- rebased §8. Wording on [N5032]

§ 1.3. Changes since R1

- added SG22 and SG6 poll results to §4. Core design
- changed the suggested alternative `BITINT_MAXWIDTH` in §4.8. Raising the `BITINT_MAXWIDTH` from 65'535 to 32'767
- expanded section on naming of the alias template after SG22 feedback
- added section on preserving integer class types and corresponding wording changes
- fixed missing return types in proposed `abs` overload and throughout the paper
- expanded section on passing `_BitInt` in functions with an observation about return types
- added §5.4. The problem of representing widths as `int`
- added section on library policy for accepting bit-precise integers in templates, and applied the proposed policy to `std::abs` overload for bit-precise integers
- updated reference from [N3699] to [N3747]
- also added `abs` overloads to `<cstdlib>`, instead of just `<cmath>`
- added changes to [utility.intcmp] (adding support to `cmp_less` was already intended, but potentially requires wording due to weird choice of constraints)

§ 1.4. Changes since R0



- updated §4.5. `_BitInt(1)` following the publication of [N3699]
- added §5.3. `make_signed` and `make_unsigned` and corresponding wording changes in § [meta.trans.sign]
- permitted bit-precise integers as underlying types of enumerations as standardized in C2y by [N3705]; see §4.3. [Underlying type of enumerations](#) and § [dcl.enum]
- further changed § [conv.prom] wording, taking promotion of enumerations with underlying bit-precise integer type into account
- added § [diff.lex] Annex C entry for the difference in the type of `0wb`
- various minor wording tweaks and added notes
- converted green notes into aqua "editor's notes" to more clearly distinguish them from wording changes

§ 2. Introduction

In distant history, there have been various attempts at standardizing multi-precision integers in C++, such as [N1692] "A Proposal to add the Infinite Precision Integer to the C++ Standard Library", [N1744] "Big Integer Library Proposal for C++0x", and [N4038] "Proposal for Unbounded-Precision Integer Types", all of which have been abandoned by the authors. However, there has always been some enthusiasm in the committee for such a feature.

I am picking up where they have left off. This effort has now converged on a C compatibility design based on fundamental types.

§ 2.1. C23

Recently, WG14's [N2763] introduced the `_BitInt` set of types to the C23 standard, and [N2775] further enhanced this feature with literal suffixes. For example, this feature may be used as follows:

```
// 8-bit unsigned integer initialized with value 255.  
// The literal suffix wb is unnecessary in this case.  
unsigned _BitInt(8) x = 0xFFwb;
```

In short, the behavior of these *bit-precise integers* is as follows:

- No integer promotion to `int` takes place.
- Mixed-signedness comparisons, implicit conversions, and other permissive features are supported.
- They have lower conversion rank than standard integers, so an operation between `_BitInt(8)` and `int` yields `int`, as does an operation with `_BitInt(N)` where N is the width of `int`. They only have greater conversion rank when their width is greater.

- Widths up to `BITINT_MAXWIDTH` are allowed, with padding bits being added if needed. `BITINT_MAXWIDTH` is at least 64.



§ 2.2. P3140R0 "`std::int_least128_t`"

In parallel, I proposed [P3140R0] which would add 128-bit integers as `std::int_least128_t` to the C++ standard. It became apparent to me that standardizing just a single width of 128 and not solving the `_BitInt` C compatibility problem would be futile, so I've stepped away from the proposal. However, the feedback and experience gained from P3140 made it well worth the time spent.

§ 2.3. P3639R0 "The `_BitInt` Debate"

I've subsequently proposed [P3639R0] "The `_BitInt` Debate", which shifts the goal to compatibility with C's `_BitInt` type, and attempts to answer whether the set of types corresponding to `_BitInt` should be a class template or a family of fundamental types. P3639R0 received much feedback in 2025. First, from SG22:

”

The WG14 delegation to SG22 believes that the C++ type family that deliberately corresponds to `_BitInt` (perhaps via compatibility macros) should be... (Fundamental/Library)

SF	F	N	L	SL
8	1	1	0	0

WG21

SF	F	N	L	SL
4	5	0	0	0

The overall sentiment in SG22 was that a fundamental type is "inevitable". This is reflected in the polls. SG6 also saw the paper, but had no clear opinion on the fundamental/library problem. Last but not least, EWG also saw the paper in Sofia 2025, with the following two polls:

”

P3639R0: EWG prefers that `_BitInt`-like type be a FUNDAMENTAL TYPE (in some form) in C++.

SF	F	N	A	SA
13	9	9	5	4

Result: consensus

P3639R0: EWG prefers that `_BitInt`-like type be a LIBRARY TYPE (in some form) in C++.

SF	F	N	A	SA
8	9	14	8	3



§ 3. Motivation

§ 3.1. Computation beyond 64 bits

Computation beyond 64 bits, such as with 128-bit integers, is immensely useful. A large amount of motivation for 128-bit computation can be found in [P3140R0]. Computations in cryptography, such as RSA, require even 4096-bit integers.

Even when performing most operations using 64-bit integers, there are certain use cases where temporarily, twice the width is needed. For example, the implementation of `linear_congruential_engine<uint64_t>` requires the user of 128-bit arithmetic, as does arithmetic with 64-bit [fixed-point numbers](#) (e.g. Q32.32).

§ 3.2. Cornerstone of standard library facilities

There are various existing and possible future library facilities that would greatly benefit from an N-bit integer type:

- As mentioned above, the implementation of `linear_congruential_engine<uint64_t>` requires the use of 128-bit integers.
- `bitset` has constructors taking `unsigned long long` and a `to_ullong` member function that converts from/to integers. This is clunky and limited considering that bitsets can be much larger than `unsigned long long`. Bit-precise integers would be a superior alternative to `unsigned long long` here.
- [P3161R4] proposes library features such as `add_carry` or `mul_wide` which produce a wider integer result than the operands. For example:

```
template<class T>
struct mul_wide_result {
    T low_bits;
    T high_bits;
};
template<class T>
constexpr mul_wide_result<T> mul_wide(T x, T y) noexcept;
```

Proposals like these are arguably obsolete if the same operation can be expressed by simply casting the operands to an integer with double the width prior to the multiplication.

§ 3.3. C ABI compatibility

C++ currently has no portable way to call C functions such as:


```
_BitInt(32) plus( _BitInt(32) x, _BitInt(32) y);  
_BitInt(128) plus(_BitInt(128) x, _BitInt(128) y);
```



While one could rely on the ABI of `uint32_t` and `_BitInt(32)` to be identical in the first overload, there certainly is no way to portably invoke the second overload.

This compatibility problem is not a hypothetical concern either; it is an urgent problem. There are already targets with `_BitInt` supported by major compilers, and used by C developers:

Compiler	BITINT_MAXWIDTH	Targets	Languages
clang 16+	8'388'608	all	C & C++
GCC 14+	65'535	64-bit only	C
MSVC	✗	✗	✗

§ 3.4. Resolving issues with the current integer type system

`_BitInt` as standardized in C solves multiple issues that the standard integers (`int` etc.) have. Among other problems, integer promotion can result in unexpected signedness changes.



EXAMPLE: The following code has undefined behavior if `int` is a 32-bit signed integer (which it is on many platforms).

```
uint16_t x = 65'535;  
uint16_t y = x * x;
```

During the multiplication `x * x`, `x` is promoted to `int`, and the result of the multiplication `4'294'836'225` is not representable as a 32-bit signed integer. Therefore, signed integer overflow takes place given unsigned operands.



EXAMPLE: The following code may have surprising effects if `std::uint8_t` is an alias for `unsigned char` and gets promoted to `int`.

```
std::uint8_t x = 0b1111'0000;  
std::uint8_t y = ~x >> 1; // y = 0b1000'0111
```

Surprisingly, `y` is not `0b111` because `x` is promoted to `int` in `~x`, so the subsequent right-shift by `1` shifts one set bit into `y` from the left. Even more surprisingly, if we had used `auto` instead of `std::uint8_t` for `y`, `y` would be `-121`, despite our code seemingly using only unsigned integers.

Overall, the current integer promotion semantics are extremely surprising and make it hard to write correct code involving promotable unsigned integers. Promotion also makes it hard to expose small integers (e.g. 10-bit unsigned integer) that exist in hardware (e.g. FPGA) in the language, since all operations would be performed using `int`. Unconventional hardware such as FPGAs are a pillar of the motivation for `_BitInt` laid out in [N2763].

§ 3.5. Portable exact-width integers



There is no portable way to use an integer with exactly 32 bits in standard C++. `int_least32_t` and `long` may be wider, and `int32_t` is an optional type alias which only exists if such an integer type has no padding bits. Having additional non-padding bits may be undesirable when implementing serialization, networking, etc. where the underlying file format or network protocol is specified using exact widths.

While most platforms support 32-bit integers as `int32_t`, their optionality is a problem for use in the standard library and other ultra-portable libraries. There are many use cases where padding bits would be an acceptable sacrifice in exchange for writing portable code, and bit-precise integers fill that gap in the language.

§ 4. Core design

The overall design strategy is as follows:

- The proposal is a C compatibility proposal first and foremost. Whenever possible, we match the behavior of the C type.
- The goal is to deliver a minimal viable product (MVP) which can be integrated into the standard as quickly as possible. This gives us plenty of time to add standard library support wherever desirable over time, as well as other convenience features surrounding `_BitInt`.

The first of these points was discussed in great detail in SG22 and SG6, and has *unanimous* support from both groups; feedback from SG22 was given 2025-10-09 during a telecon:

”

/Poll/: Do you agree with the author's position on fundamental types being better than class template for `_BitInt`>?

Any objections to unanimous consent? /None/

/Poll/: Do you agree with allowing `0wb = _BitInt(1)` and `enum E : _BitInt(N)`, assuming C adopts N3699 and N3705?

Any objections to unanimous consent? /None/

/Poll/: Do you agree with keeping UB on signed integer overflow for `_BitInt`?

Any objections to unanimous consent? /None/

/Poll/: Do you agree that WG21 keep all implicit conversions for `_BitInt`?

Any objections to unanimous consent? /None/

/Poll/: Do you agree that WG21 keep the lower limit on the value of `BITINT_MAXWIDTH` from C?

Any objections to unanimous consent? /None/

/Poll/: Do you agree that WG21 should add a `_BitInt` keyword?

Any objections to unanimous consent? /None/

[...]

Group agrees that we want to pursue compatibility between C and C++ with regards to `_BitInt`



Both directions mentioned in that poll have since been adopted by C2y, via [N3747] and [N3705].

SG6 had concerns regarding the standard library impact of bit-precise integers, but agreed with the core design strategy during the Kona 2025 meeting:

”

POLL: Let `_BitInt` have the exact same semantics as in C.

SF	F	N	A	SA
7	2	0	0	0

- **Author Position:** SF
- **Outcome:** Strong consensus in favor



NOTE: The use of "C" in the above SG6 poll is somewhat ambiguous. The issues of `_BitInt(1)` and bit-precise underlying enumeration types were presented to SG6, and SG6 seemed to agree with the author's choices once it was clear that C2y is heading in this direction anyway.

Overall, both SG22 and SG6 agree that `_BitInt` in C++ should match the C design, and keeping it in sync with C2y's changes since C23 is necessary for that.

EWG then reaffirmed every decision with resounding consensus during the 2026-03 Croydon meeting:

”

Forward P3666R3 to CWG and LEWG for C++29

SF	F	N	A	SA
17	29	3	2	0

Result: strong consensus in favor

§ 4.1. Why not a class template?

[P3639R0] explored in detail whether to make it a fundamental type or a library type. Furthermore, feedback given by SG22 and EWG was to make it a fundamental type, not a library type. This boils down to two plausible designs (assuming `_BitInt` is already supported by the compiler), shown below.

F – Fundamental type	L – Library type
<pre>template <size_t N> using bit_int = _BitInt(N);</pre>	<pre>template <size_t N> class bit_int { private:</pre>



```
template <size_t N>
using bit_uint =
    unsigned _BitInt(N);
```

```
_BitInt(N) _M_value;
public:
    // ...
};
template <size_t N>
class bit_uint
{ /* ... */; };
```

The reasons why we should prefer the left side are described in the following subsections.

§ 4.1.1. LEWG is not convinced that a library type should be pursued

During the 2026-03 Croydon meeting, the following poll was taken:

”

POLL: We should provide `std::bit_int` and `std::bit_uint` as class templates (not necessarily as part of P3666)

SF	F	N	A	SA
2	4	8	4	0

Author's Position: WA

Outcome: No consensus



NOTE: This poll was taken after `_BitInt` was already accepted as a fundamental type by EWG, so the result is asking whether both a fundamental type and a library type should be pursued, rather than putting the options against each other.

§ 4.1.2. Full C compatibility requires fundamental types

`_BitInt` in C can be used as the type of a bit-field, among other places:

```
struct S {
    // 1. _BitInt as the underlying type of a bit-field
    _BitInt(32) x : 10;
};

// 2. _BitInt in a switch statement
_BitInt(32) x = 10;
switch (x) {}

// 3. _BitInt used as a null pointer constant
void* p = 0wb;

// 4. _BitInt used as underlying type of enumeration
// (not valid in C23, but valid in C2y)
enum S : _BitInt(32) { X = 0 };
```

Since C++ does not support the use of class types in bit-fields, such a `struct S` could not be passed from C++ to a C API. A developer would face *severe* difficulties when porting C code

which makes use of these capabilities to C++ and if bit-precise integers were a class type in C++.



§ 4.1.3. Common spelling of unsigned `_BitInt(N)`

If bit-precise integers were class types in C++, this would cause a serious problem with a common spelling that can be used in both C and C++ headers, even if there was a `_BitInt` compatibility macro.

```
#define _BitInt(...) std::bit_int<__VA_ARGS__>

unsigned _BitInt(8) x; // error: cannot combine 'unsigned' with class type
```

There are some workarounds to the problem, but they all seem unattractive:

- Permitting `signed` and `unsigned` to be combined with class types in general, perhaps with the effect of applying `std::make_signed` and `std::make_unsigned`. This would lead to a bifurcation of the language where both a builtin feature and a type trait achieve the same effect.
- "Blessing" the `std::bit_int<N>` *type-name* so it can be combined with `unsigned`. This would be a highly unusual special case in the language.
- Making `_BitInt(...)` expand to an unspecified construct that can be combined with `signed` and `unsigned`. This means there needs to be a fundamental type, although that fundamental type only acts as a proxy for the `std::bit_int` class type. Once again, this comes off as an unusual special case.
- Introducing a `_BitUInt(...)` macro for unsigned bit-precise integers, and insisting that both C and C++ developers use this for interoperability. This feels like an unnecessary burden for C developers considering that their spelling works perfectly fine and that we have other design options which keep C code intact.

§ 4.1.4. C compatibility would require an enormous amount of operator overloads etc.

Integer types can be used in a large number of places within the language. If we wanted a `std::bit_int` class type to be used in the same places (which would be beneficial for C-interoperable code), we would have to add a significant amount of operator overloads and user-defined conversion functions:

- There are conversion to/from floating-point types and other integral types.
- There are conversion to/from enumeration types.
- There are conversion to/from pointers, at least for `_BitInts` of the same width as `uintptr_t`.
- Integers can be used to add offsets onto pointers, and by proxy, in the subscript operator of builtin arrays.
- Arithmetic operators can be used to operate between any mixture of arithmetic types, such as `_BitInt(32) + float`.

Any discrepancies would lead to some code using bit-precise integers behaving differently in C and C++, which is undesirable.

Furthermore, the *wb integer-suffix* for `_BitInt` is fairly complicated to implement as a library feature because the resulting type depends on the numeric value of the literal. This means it would presumably be implemented like:

```
template<char... Chars>
constexpr auto operator""wb() { /* ... */ }
template<char... Chars>
constexpr auto operator""WB() { /* ... */ }
template<char... Chars>
constexpr auto operator""uwb() { /* ... */ }
template<char... Chars>
constexpr auto operator""UWB() { /* ... */ }
template<char... Chars>
constexpr auto operator""uWB() { /* ... */ }
template<char... Chars>
constexpr auto operator""Uwb() { /* ... */ }
```

Seeing that properly emulating C's behavior for `_BitInt` (and its suffixes) requires a mountain of complicated operator overload sets, user-defined conversion functions, converting constructors, and user-defined literals, it seems unreasonable to go this direction.

A major selling point of a library type is that library types have more teachable interfaces, since the user simply needs to look at the declared members of the class to understand how it works. If the interface is a record-breaking convoluted mess, this benefit is lost. If we choose not to add all this functionality, then we lose a large portion of C compatibility. Either option is bad, and making `std::bit_int` a fundamental type seems like the only way out.

§ 4.1.5. Constructors cannot signal narrowing

Some C++ users prefer list initialization because it prevents narrowing conversion. This can prevent some mistakes/questionable code:

```
unsigned x = -1; // OK, x = UINT_MAX, but this looks weird
unsigned y{ -1 }; // error: narrowing conversion
```

This would not be feasible if `std::bit_int` was a library type because narrowing cannot be signaled by constructors. Consider that `std::bit_int` and `std::bit_uint` should have a non-explicit constructor (template) accepting `int` (and other integral types) to enable compatibility in situations like:

```
#ifdef __cplusplus
typedef std::bit_uint<32> u32; // C++
#else
typedef unsigned _BitInt(32) u32; // C
#endif
// Common C and C++ code, possibly in a header:

// OK, converting int → u32.
// Using "incorrectly typed" zeros is fairly common, both in C and in C++.
u32 x = 0;
// OK, same conversion, but would be considered narrowing in C++.
```

```
// Not very likely to be written.  
u32 y = -1;
```



If such a `std::bit_uint<32>(int)` constructor existed, the following C++ code would not raise any errors:

```
std::bit_uint<32> x{ 0 }; // OK, as expected  
std::bit_uint<32> y{ -1 }; // OK?! But this looks narrowing!
```

This code simply calls a `std::bit_uint<32>(int)` constructor, and while the initialization of `y` is *spiritually* narrowing, no narrowing conversion actually takes place. In conclusion, if `std::bit_uint` was a library type, C++ users who use this style would lose what they consider a valuable safety guarantee.



DRAFTING NOTE: It can be argued that using list-initialization for this purpose is an anti-pattern and only solves a subset of the issues that compiler warnings and linter warnings should address. Personally, I have no strong position on this issue.

§ 4.1.6. Tiny integers are useful in C++

In some cases, tiny `_BitInt` types may be useful as the underlying type of an enumeration:

```
enum struct Direction : _BitInt(2) {  
    north, east, south, west,  
};
```

By using `_BitInt(2)` rather than `unsigned char`, every possible value has an enumerator. If we used e.g. `unsigned char` instead, there would be 252 other possible values that simply have no name, and this may be detrimental to compiler optimization of `switch` statements etc.



DRAFTING NOTE: See also §4.3. Underlying type of enumerations.

§ 4.1.7. Special deduction rules

While this proposal focuses on the minimal viable product (MVP), a possible future extension would be new deduction rules allowing the following code:

```
template <size_t N>  
void f(_BitInt(N) x);  
  
f(_BitInt(32)(0)); // calls f<32>
```

Being able to make such a call to `f` is immensely useful because it would allow for defining a single function template which may be called with every possible signed integer type, while only producing a single template instantiation for `int`, `long`, and `_BitInt(32)`, as long as those three have the same width. The prospect of being able to write bit manipulation utilities that simply accept `unsigned _BitInt(N)` is quite appealing.

If `_BitInt(N)` was instead surfaced only through a class type, this would not work because template argument deduction would fail, even if there existed an implicit conversion sequence from `int32_t` to that class type. These kinds of deduction rules may be shutting the door on this mechanism forever.

§ 4.1.8. Special overload resolution rankings

Yet another possible future extension would be rankings for overload resolution that take integer width into account.



EXAMPLE: Special overload rankings could make bit-precise integers more easily interoperate with existing overload sets:

```
struct QString { // see Qt 6 documentation
    static QString number(int n, int base = 10);
    static QString number(long n, int base = 10);
    static QString number(long long n, int base = 10);
    // ...
};

QString::number(0wb); // currently ambiguous, but could call QString::nu
```

This could be valid if `number(int)` was considered a better match on the basis that its width is closer to that of `0wb`. Further disambiguation could be applied if `int` and `long` had the same width.



EXAMPLE: Special overload rankings could make it possible to create non-template overload sets that cover a greater range of widths:

```
unsigned _BitInt(64) widening_mul(unsigned _BitInt(32));
unsigned _BitInt(128) widening_mul(unsigned _BitInt(64));

widening_mul(128wb); // OK, calls widening_mul(unsigned _BitInt(64))
```

These overload ranking rules would be difficult or impossible to define using a class type. Of course, they are not proposed, and it's not certain whether such rules are desirable to have, but it would be unfortunate to shut the door on these possible features forever.

§ 4.1.9. Quality of implementation requires a fundamental type

While a library type `class bit_int` gives the implementation the option to provide no builtin support for bit-precise integers, to achieve high-quality codegen, a fundamental type is *inevitably* needed anyway.



EXAMPLE: When an integer division has a constant divisor, like `x / 10`, it can be optimized to a fixed-point multiplication, which is much cheaper:


```
unsigned div10(unsigned x) {  
    return x / 10;  
}
```



For this operation, Clang emits the following assembly:

```
div10(unsigned int):  
    mov     ecx, edi  
    mov     eax, 3435973837  
    imul    rax, rcx  
    shr     rax, 35  
    ret
```

Basically, the result is rewritten as `x * 3435973837ull >> 35`. This optimization is called *strength reduction* and may lead to dramatically faster code, especially when the hardware has no direct support for integer division. Similarly, multiplication can be strength-reduced to bit-shifting when a factor is a power of two, remainder operations can be reduced to bitwise AND when the divisor is a power of two, etc.

Performing strength reduction requires the compiler to be aware that a division is taking place, and this fact is lost when division is implemented in software, as a loop which expands to hundreds of IR instructions when unrolled.

Furthermore, the compiler frontend needs to understand certain operations to warn about obvious mistakes such as division by zero, shifting by an overly large amount, producing signed integer overflow unconditionally, etc. Use of `pre` on e.g. `bit_int::operator/` cannot be used to achieve this because numerics code needs to have no hardened preconditions and no contracts, for performance reasons.

Last but not least, a fundamental type is needed to speed up constant evaluation. Something like integer division between two `bit_int<128>` may be much faster as a compiler-builtin operation compared to constant-evaluating a "software division" loop with 128 iterations necessary to implement [binary division](#).

If we accept the premise that a fundamental type is needed anyway (possibly as an implementation detail of a class template), then the class template is actively harmful bloat:

- Any arithmetic operation needs to go through overload resolution, competing with countless other `operator`+s (there are many in the standard library already). Even if implementers special-case these operations to circumvent the (usually awful) diagnostic quality of a failed call to `operator+`, there remains substantial cost: overload resolution is expensive.
- Every distinct `bit_int<N>` and `bit_uint<N>` would be a separate instantiation of a relatively large class template, which would undoubtedly add compilation cost.
- Invocations of member functions or operator overloads may add cost to debug builds and constant evaluation.

§ 4.2. Why the `_BitInt` keyword spelling?

I also propose to standardize the keyword spelling `_BitInt` and `unsigned _BitInt`. When the `bit_int` alias template was still proposed, I considered this a "C compatibility spelling" rather than the preferred one which is taught to C++ developers. Now, it is the only spelling of bit-precise integers in this paper, which should be motivation alone.

While a similar approach could be taken as with the `_Atomic` compatibility macro, macros cannot be exported from modules, and macros needlessly complicate the problem compared to a keyword. Furthermore, to enable compiling shared C and C++ headers, all of the spellings `_BitInt`, `signed _BitInt` and `unsigned _BitInt` need to be valid. This goes far beyond the capabilities that a compatibility macro like `_Atomic` can provide without language support. If the `_BitInt(...)` macro simply expanded to `bit_int<__VA_ARGS__>`, this may result in the ill-formed code `signed bit_int<N>`.

The most plausible fix would be to create an exposition-only *bit-int* spelling to enable `signed bit-int<N>`, which makes our users raise the question:

” Why is there a compatibility macro for an exposition-only keyword spelling?! Why are we making everything more complicated by not just copying the keyword from C?! Why is this exposition-only when it's clearly useful for users to spell?!

The objections to a keyword spelling are that it's not *really* necessary, or that it "bifurcates" the language by having two spellings for the same thing, or that those ugly C keywords should not exist in C++. Ultimately, it's not the job of WG21 to police code style; the keyword spelling should be standardized for interoperability.

The `_BitInt` spelling is useful for writing C/C++-interoperable code, and C compatibility is an important design goal.

Even if compatibility macros exist in some code bases, the proposal itself should standardize the keyword spelling. Since there is no clear technical benefit to a macro, the keyword is the *only* logical choice.

i **NOTE:** Clang already supports the `_BitInt` keyword spelling as a compiler extension, so this is standardizing existing practice.

§ 4.3. Underlying type of enumerations

The following C code is not valid C23, but is valid in C2y following acceptance of [N3705].

```
// error: '_BitInt(32)' is an invalid underlying type
enum E : _BitInt(32) { x = 0 };
```

There is no obvious reason why `_BitInt` must not be a valid underlying type, neither in C nor in C++. For C++, it seems better to simply allow bit-precise integers in this context because it is useful; see §4.1.6. [Tiny integers are useful in C++](#).

Also note that as adopted in [N3705], bit-precise integers should only be the underlying types of enumerations when the user explicitly specifies this with : `_BitInt(N)`:



```
enum class E : _BitInt(1024) {
    X = 0x1'0000'0000'0000'0000'0000'0000'0000wb // OK
};

enum E {
    X = 0x1'0000'0000'0000'0000'0000'0000'0000wb // error (most likely)
};
```

As adopted in [N3705] and as in the case of bit-precise bit-fields, integer promotion should not take place for enumerations whose underlying type is bit-precise. If the implementation-defined underlying type of enumerations could be chosen to be bit-precise, this would make it implementation-defined whether integer promotion takes place, by proxy. It would also be a compatibility pitfall; C requires bit-precise underlying types to be specified explicitly, so any choice the implementation makes could interfere with future standardization.



NOTE: See [N3550] §6.7.3.3 "Enumeration specifiers" for current restrictions. Note that in C, "enumerated types" are also classified as "integer types", unlike in C++.

§ 4.4. Should bit-precise integers be optional?

As in C, `_BitInt(N)` is only required to support `N` of at least `LLONG_WIDTH`, which has a minimum of 64. This makes `_BitInt` a semi-optional feature, and it is reasonable to mandate its existence, even in freestanding platforms.

Of course, this has the catch that `_BitInt` may be completely useless for tasks like 128-bit computation. As unfortunate as that is, the MVP should include no more than C actually mandates. Mandating a greater minimum width could be done in a future proposal.

§ 4.5. `_BitInt(1)`

C23 does not permit `_BitInt(1)` but does permit `unsigned _BitInt(1)`, mostly for historical reasons (C did not always require two's complement representation for signed integers). This is an irregularity that could make generic programming harder in C++.

However, this restriction is being lifted in C2y; see [N3747] "Integer Sets, v5". That proposal has been approved but not yet merged into the C2y draft at the time of writing. It makes `_BitInt(1)` a valid type, and `0wb` is changed to be of type `_BitInt(1)` rather than `_BitInt(2)`. It also contains some practical motivation for why a single-bit should be permitted.



NOTE: If `_BitInt(1)` was allowed, it would be able to represent the values 0 and -1, just like an `int x : 1`; bit-field.

§ 4.6. Undefined behavior on signed integer overflow

I propose to perpetuate bit-precise integers having undefined behavior on signed integer overflow, just like `int`, `long` etc. This has a few reasons:



- bit-precise integers have undefined overflow in C, so this is what users are used to.
- "Solving" signed integer overflow for bit-precise integers is not part of the MVP. Undefined behavior can always be defined to do something else, so there is no urgent need for this paper to address this issue, rather than solving it in a follow-up paper.
- Signed integer overflow having undefined behavior is a much broader issue that should be looked at in general, for all integer types, not just bit-precise integer types. Perhaps hardened implementations could have wrapping overflow with erroneous behavior. In any case, the problem exceeds the scope of the paper.
- It is highly unusual that users would expect signed integer overflow to be well-behaved, such as having wrapping behavior. Adding two positive numbers and obtaining a negative number is not typically useful.
- The undefined behavior here is useful. It allows for optimizations such as converting `x + 3 < 0` into `x < -3`.

That being said, much of the feedback surrounding bit-precise integers revolved around signed integer overflow. If we were to make signed integer overflow *not* undefined for bit-precise integers, there are two options that may find consensus:

- Make signed integer overflow wrapping. In other words, most operations would be performed as if by casting to the corresponding unsigned type, performing the operation, and casting back.
- Make signed integer overflow wrapping *and* erroneous. This is mirroring Rust's behavior, and would typically be implemented by detecting overflow on debug builds and in constant evaluation, but ignoring it and letting it wrap in release builds.

§ 4.7. Permissive implicit conversions

Just like any other integral type, the proposal makes bit-precise integers quite permissive when it comes to implicit conversions. This is disappointing to anyone who wants bit-precise integers to be a much "stricter" or "safer" alternative to standard integers, but it is arguably the better design for various reasons.

§ 4.7.1. C compatibility

Firstly, the point of perpetuating implicit conversions is to mirror the C semantics as closely as possible, which leads to few or no surprises when porting code between the languages, or when writing C-interoperable headers.

If we look at how C users use `_BitInt`, [GitHub code search for "_BitInt" language:C](#) yields examples such as:

”

```
// mixing signed and unsigned bit-precise integers
unsigned _BitInt(128) max128s = 0x7FFF'FFFF'FFFF'FFFF'FFFF'FFFF'FFFF'FFFF
// mixing bit-precise and standard integers
```

```
unsigned _BitInt(4) a = 1u;  
// mixing bit-precise and standard integers of different signedness  
unsigned _BitInt(total) bit = 1;  
// ... including cases where initialization does not preserve values  
unsigned _BitInt(3) max3u = -1;
```



If we were to make implicit conversions much more restrictive on the C++ side, it would become very easy to slip up and accidentally write a header that does not also compile in C++.

§ 4.7.2. Difficulty of carving out exceptions in the language

Writing C++ code involving bit-precise integers would be quite annoying and "flag" many harmless cases if the rules were too strict.



EXAMPLE: The following line of code would not compile if converting from `int` to `unsigned _BitInt(8)` was unconditionally ill-formed.

```
unsigned _BitInt(8) x = 0; // error?
```

`0` is "incorrectly signed" for `unsigned _BitInt(8)`, and the conversion from `int` to `unsigned _BitInt(8)` is not value-preserving generally, but writing code like this is perfectly reasonable.

The workaround would be to use correct literals, such as:

```
unsigned _BitInt(8) x = 0uwb; // OK, conversion unsigned _BitInt(1) → un...
```

To combat this problem, it would be necessary to carve out various special cases. For example, permitting value-preserving conversions with constant expressions would prevent the example above from being flagged.



NOTE: There is precedent for such special casing of value-preserving conversions. Specifically, see mentions of "narrowing" in [\[dcl.init.list\]](#), [\[expr.spaceship\]](#), and [\[expr.const\]](#).

However, such special cases are insufficient to cover *all* harmless cases.



EXAMPLE:

```
void for_each_cell(vec3 x) {  
    for (int i = 0; i < 3; ++i) {  
        do_something(x[i]);  
    }  
}
```

Even though `i` is not a constant expression, `x[i]` will "just work" no matter what integer type `vec3::operator[]` accepts.

Existing C++ code bases that have not used flags such as `-Wconversion` from the start are likely filled with many such harmless cases of mixed-sign implicit conversions. If bit-precise integer types were introduced into these code bases, refactoring effort may be unacceptable.

Furthermore, discrepancies between the standard integers and bit-precise integers would make it much harder to write generic code:



EXAMPLE: The following function template may be instantiated with any integral type `T`, but the instantiation would be ill-formed for `T = unsigned _BitInt(8)` with restrictive implicit conversions:

```
template<std::integral T>
T div_ceil(T x, T y) { // performs integer division, rounding to +inf
    // ⚠ Could be mixed-sign comparison:
    bool quotient_positive = (x ^ y) >= 0;
    // ⚠ Could be mixed-sign comparison
    bool adjust = x % y != 0 && quotient_positive;
    // ⚠ Could be mixed-sign addition between int (0 or 1)
    // and unsigned _BitInt(N) "x / y":
    // ⚠ Could be lossy conversion when returning: int → unsigned _BitInt
    return x / y + int(adjust);
}
```

Literally every statement of this template may fail to compile when `T = unsigned _BitInt(8)`, depending on how strict implicit conversions are. I conjecture that there are vast amounts of templates like `div_ceil`. To accommodate bit-precise integers in this function, a rewrite is necessary:

```
template<std::integral T>
T div_ceil(T x, T y) {
    constexpr auto zero = T(0);
    bool quotient_positive = (x ^ y) >= 0 & zero;
    bool adjust = x % y != 0 & zero && quotient_positive;
    return x / y + int T(adjust);
}
```



EXAMPLE: The following function template involves a mixed-sign operation, but is entirely harmless for any type `T`:

```
constexpr unsigned mask = 0xf;
T x = /* ... */;
x &= mask; // equivalent to x = x & mask;
```

Even if `x` is signed instead of unsigned, `x & mask` produces a mathematically identical result.

While conversions between bit-precise integers and other signed or unsigned integer could be difficult to restrict due to the reasons above, other conversions are much more rare and could more easily be restricted:

- Conversions between bit-precise integers and `bool`.
- Conversions between bit-precise integers and character types.
- Conversions between bit-precise integers and floating-point types.

It would be reasonable to ban these conversions unconditionally because they are likely to be category errors.



BUG: Consider the "easter egg" discovered in cplusplus.com/forum/general/105627/:

”

I was fixing a couple of minor bugs in a program I've been working on, when I made the mistake of typing `cout<<string('\n', 1);` instead of `cout<<string(1, '\n');`

I didn't get any compile errors and the programs reaction gave me a bit of a laugh. Instead of the blank line I wanted to put in, I got `:):):):):):):):):):)` (10 of them). It just made me wonder as a relative C++ beginner what other "easter eggs" are there that people might feel like sharing.

It turns out that `string('\n', 1)` is not an "easter egg", it just results in the Windows terminal displaying a `char(1)` as `":)"` ten times. The `string(size_t, char)` overload is called, and since the `'\n'` and `1` can be converted to `size_t` and `char` without any change in value, compilers generally don't raise a warning, even with `-Wconversion` enabled.

The least harmful of these conversions is a value-preserving conversion from a bit-precise integer to a floating-point type. However, at best, these lack clarity of intent.



EXAMPLE: Consider a code base with the following two functions computing $\sqrt{x}$:

```
int isqrt(int x);
double sqrt(double x);
```

When the user calls `sqrt` with an integer operand, are we sure that this decision was made intentionally? Is the author unaware that there is a separate function giving the integer results, or do they actually need the fractional part, and that is why they called the `double` overload? Even if the author wrote `(int) sqrt(/* ... */)`, this could plausibly be done due to performance considerations.

Similarly, calling `std::sqrt` with an integer operand could be a major performance bug on a 32-bit platform with 32-bit `float` and 64-bit `double`, considering that this is equivalent to calling `std::sqrt(double)`. Perhaps calling `std::sqrt(float)` was intended.

Conversely, if the author called `isqrt(10.f)`, the `float` $\rightarrow$ `int` conversion may be value-preserving, but this call is almost certainly a mistake. The author likely expected to obtain `3.1623f`, judging by the operand.

§ 4.7.4. Conclusion on implicit conversions

In conclusion, discrepancies between the standard integers and bit-precise integers are undesirable; they introduce a lot of unnecessary problems. There are many harmless operations like `T x = 0;` and `x & mask` where mixing signedness is okay, and not every user wants to have warnings, let alone errors for these. Especially errors would make it hard to write headers that compile both in C and in C++.

The final nail in the coffin is that if the user wants implicit conversions to be restricted, they have the freedom to add those restrictions via compiler warnings and linter checks. Having these restrictions standardized in the language robs the user of choice. If C++26 profiles make progress, it is likely that C++ will have profiles which restrict implicit conversions, giving users a standard way to opt into diagnostics.

This revision keeps implicit conversions permissive. If desired, conversions described in §4.7.3. [Picking some low-hanging fruits](#) can still be restricted in a follow-up paper.

§ 4.8. Raising the `BITINT_MAXWIDTH`

The proposal currently does not seek to increase the `BITINT_MAXWIDTH` beyond what C offers. That is, `BITINT_MAXWIDTH` may be as low as 64. I do not consider an increase of the maximum to be part of the MVP. It's something that can always be done later, if desirable, without any breaking changes.

It also should be stated that increasing the `BITINT_MAXWIDTH` is not *really* within the power of WG21 and not even within the power of compiler vendors.



EXAMPLE: Clang supports a `BITINT_MAXWIDTH` of up to `8'388'608`, but only enables this for certain ABIs. For example, the `x86-64 psABI` defines an ABI for any bit-precise integer width, so the full width is available.

However, the "Basic" C ABI for WebAssembly (which Clang uses at the time of writing) has the following limitation:

” `_BitInt(N)` types are supported up to width 128 and are represented as the smallest same-signedness Integer type with at least as many bits.

Consequently, `BITINT_MAXWIDTH` is set to `128` when compiling with `--target=wasm32-unknown-unknown`.

WG21 can define the `BITINT_MAXWIDTH` as whatever they want to; it is of no consequence because compiler vendors are not going to make that width available when there is no platform ABI for `_BitInt(BITINT_MAXWIDTH)`. If compiler vendors did that, there would be a risk of a massive future ABI break in order to comply with the system ABI, once defined. Without a

single platform ABI, there would also be no portable way for code generated by different compilers to interoperate, such as compiling a C library with GCC and using it from Clang-compiled C++ code.

An increase to the `BITINT_MAXWIDTH` is political posturing. That does not mean that it's entirely pointless. If C++ defined the minimum to be, say, `32'767`, this would motivate platforms to define an ABI for large bit-precise integers.

§ 4.8.1. Possible increased `BITINT_MAXWIDTH` values

Firstly, it should be noted that [\[P3140R0\]](#) got substantial criticism just for attempting to standardize 128-bit integers for embedded developers. As a compromise, it may be reasonable to increase the `BITINT_MAXWIDTH` only for hosted implementations, not for freestanding implementations. That being said, there are two plausible increased minimums:

- `128`. Many platform ABIs (see example above) already define an ABI for `_BitInt(128)`. 128-bit integers have been provided by compilers for a long time now, at least by GCC and Clang (`__int128`). There are heaps of motivation (see [\[P3140R0\]](#)) for 128-bit computation. The calling conventions are also relatively obvious for 64-bit platforms: pass via pair of 64-bit integers.
- `32'767`. Both GCC and Clang already support this width. Some cryptographic use cases like future-proof RSA computations need 8192 bits of key size, and at least double that for modular arithmetic. It is unlikely that a cryptographic library needs 4096 bits but does not need 8192 bits at any point, but likely that 32767 is sufficiently large, even in the next few years. Any more than 32767 becomes problematic for the standard library because `int` is no longer capable of representing the width on 16-bit platforms; this breaks functions such as `std::popcount` (which returns `int`). Major design adjustments would be needed to address this problem.

Beyond that, `_BitInt` may be tricky to use. When working with Clang's `_BitInt(8'388'608)`, a single `+` operation could result in stack overflow because the result is 1 MiB large. The user would have to carefully ensure that all objects (including temporaries) have static or dynamic storage duration (i.e. use `new` or global variables). For these extreme sizes, a dynamically sized integer is more ergonomic. Therefore, setting the minimum to millions feels unmotivated.

§ 4.9. Template argument deduction

The following code should be valid:

```
template <std::size_t N>
    void f(_BitInt(N));
template <auto N>
    void g(_BitInt(N));

int main() {
    f(_BitInt(3)(0)); // OK, N = 3
    g(_BitInt(3)(0)); // OK, N = 3, where N is of type std::size_t
}
```

This would be a consequence of deduction from `_BitInt` being valid:



```
template <unsigned N>
    void f(_BitInt(N));
template <int N>
    void g(_BitInt(N));

int main() {
    f(_BitInt(3)(0)); // OK, N = 3
    g(_BitInt(3)(0)); // OK, N = 3
}
```

This behavior is already implemented by Clang as a C++ compiler extension, and makes deduction behave identically to deducing sizes of arrays. In general, the aim is to make the deduction of `_BitInt` widths as similar as possible to arrays because users are already familiar with the latter. It is also clearly useful because it allows writing templates that can accept `_BitInt` of any width.

This behavior is part of the core design, and it would be quite surprising to users if such deduction was not possible. If deducing `N` from `std::array<T, N>` is possible, why would it not be possible to deduce `N` from `_BitInt(N)`?

One thing deliberately not allowed is:

```
_BitInt x = 123wb;
```

This shorthand construct (which is similar to class template argument deduction) is not part of the MVP and if desired, should be proposed separately.

§ 4.10. No preprocessor changes, for better or worse

To my understanding, no changes to the preprocessor are required. [N2763] did not make any changes to the C preprocessor either. In most contexts, integer literals in the preprocessor are simply a *pp-number*, and their numeric value or type is irrelevant.

Within the controlling constant expression of an `#if` directive, all signed and unsigned integer types behave like `intmax_t` and `uintmax_t` ([cpp.cond]), which may be surprising.



EXAMPLE: The following code is ill-formed if `intmax_t` is a 64-bit signed integer (which it is on many platforms):

```
#if 1'000'000'000'000'000'000'000'000'000wb // error
#endif
_BitInt(81) x = 1'000'000'000'000'000'000'000'000'000wb; // OK
```

`#if 1'000'000'000'000'000'000'000'000'000wb` is ill-formed because the integer literal is of type `_BitInt(81)`, which behaves like `intmax_t` within `#if`. Since 10^{32} does not fit within `intmax_t`, the literal is ill-formed ([lex.icon] paragraph 4).

The current behavior could be seen as suboptimal because it makes bit-precise integers dysfunctional within the preprocessor. However, the preprocessor is largely "owned" by C, and any fix should go through WG14. In any case, fixing the C preprocessor is not part of the MVP. 

§ 4.11. Padding in `_BitInt`

It is worth mentioning that `_BitInt` types may have padding bits, which the implementation can avoid for standard integer types by choosing padding-free widths for them. This is a known fact, and there is no desire to prohibit `_BitInt` widths that would have padding bits.

A possible future direction could be to mandate sign extension for those padding bits. The standard currently does not mandate padding bits to have specific values in most cases, but that may be useful for `_BitInt`.



EXAMPLE: `_BitInt(3)` could be correctly converted to `_BitInt(CHAR_BIT)` using `std::memcpy` if sign extension in the padding bits was guaranteed. That is, the conversion would have the same effect as `static_cast`.

The problem with such a guarantee is that C does not provide it, so when calling an `extern "C"` function that takes e.g. `_BitInt(3)*` as a parameter, it would be impossible for a C++ program to guarantee that the padding bits have been correctly set by the C program. This guarantee can only be provided by C and C++ in tandem, is fairly ambitious, requires ABI changes, and needs to be proposed separately.

§ 5. Library design

In summary, the design for the standard library is to support `_BitInt` where C already supports it, and to support it in `<type_traits>`. Anywhere else, support is *prevented*.

§ 5.1. Preventing library support for `_BitInt`

When discussing library design, it is important to understand that the vast majority of support for bit-precise integers "sneaks" into the standard without any explicit design changes or wording changes. This happens because bit-precise integers are proposed to be signed and unsigned integer types, so they would be supported by any facility that supports integer types (e.g. `<bit>`).

Even if the wording effort to support bit-precise integers is minimal in some cases, and even if the implementation effort boils down to adjusting a template constraint, such implicit `_BitInt` support results in an explosion of the test matrix. For example, `std::to_chars` may be implemented generically for all integer types, but tests still need to be written to ensure that it works for any `_BitInt(N)`. Furthermore, R3 of this paper hit several design problems, like whether to still pass huge `_BitInt` by value and how to extend functions like `std::to_chars` or `std::to_string` that are not templates in the standard library, but overload sets of non-

template functions. Trying to word and implement support for `_BitInt` all in one paper is simply too much.



LEWG addressed this concern as follows:

”

POLL: We should prevent library support for `_BitInt`

SF	F	N	A	SA
5	9	2	3	1

Outcome: consensus in favor

Since LEWG voted to *prevent* library support for bit-precise integers, we must alter the constraints of various library components to disallow bit-precise integers. To be clear, this does not mean that we prevent `std::vector<_BitInt(N)>` or anything else that works generically for copyable types, movable types, etc. Such a restriction would be totally toothless anyway because it can be bypassed by wrapping `_BitInt` in a `struct`. Only numeric constraints are adjusted.

The interesting question is where we do support bit-precise integers despite the LEWG vote:

- `std::is_integral_v<_BitInt(N)>` is `true`, directly contradicting an LEWG poll
- `std::make_signed` and `std::make_unsigned` support bit-precise integers
- `std::numeric_limits<_BitInt(N)>` is specialized
- `<cmath>` functions support bit-precise integers as arguments
- `<stdbit.h>` functions support bit-precise integers in those cases where C23 supports them too



NOTE: Preventing library support right now does not preclude supporting `_BitInt` throughout the standard library in the future. The strategy for C++29 can still be to add support like `<charconv>`, `<bit>`, `<format>`, etc. paper-by-paper.

§ 5.2. Broadening `is_integral`

One controversial aspect of this paper is that `std::is_integral_v<_BitInt(N)>` is proposed to be `true`. LEWG voted for the opposite during the 2026-03 Croydon meeting:

”

POLL: `std::is_integral_v<_BitInt(N)>` should be false

SF	F	N	A	SA
7	4	5	2	0

Outcome: consensus in favor

This decision was motivated by the fact that otherwise, large amounts of existing code (not just the standard library, but also user code) would implicitly opt into supporting bit-precise

integers, despite never being written with that intent. It is even theoretically possible that this results in “correctness regression” due to instantiating templates with e.g. `_BitInt(1)` and running into signed integer overflow that would have been prevented by promotion to `int`. 

Nonetheless, this poll result caused widespread backlash from members of the committee, Clang implementers, and the C++ community at large. Some described the decision as “absurd”. There are also many practical problems not discussed during the LEWG session which likely would have prevented consensus from being reached.

§ 5.2.1. Reasons for making `std::is_integral_v<_BitInt(N)> true`

1. First and foremost, it is intuitive for bit-precise integers to be integral types. This is what users expect.
2. A substantial amount of committee members don't believe LEWG is entitled to a decision on this type trait in the first place. That is because the type trait arguably just exposes the fact that `_BitInt` is an integral type within the core language, so only EWG, not LEWG can decide the result of `std::is_integral`; LEWG does not decide what is an integral type in the core language. I personally don't believe that LEWG is powerless in this regard, but the LEWG decision does cause procedural controversy.
3. Classifying `_BitInt` as an integral type is symmetrical with the taxonomy of integer types in C, where bit-precise integer types are integer types.
4. Any code exclusively constrained on `std::is_integral_v` is likely under-constrained anyway because it opts into `char32_t`, `const volatile bool`, and other types that might not match the user's expectation of “integral types”. `_BitInt` would make this problem worse, but it does not create a new problem.
5. `libc++` already makes `std::is_integral_v<_BitInt(N)> true`, so what LEWG wants is silently altering the behavior of existing traits. If LLVM implementers have already shipped this behavior in production code, despite the potential impact on code constrained on `std::is_integral_v`, why is LEWG effectively claiming it's not feasible to be shipped due to user impact?
6. For the most common use cases of bit-precise integers, such as `_BitInt(128)`, there really isn't a problem caused by implicit support in the standard library or in third-party code. In fact, `std::is_integral_v<__int128>` has already been decided to be true by `libstdc++` and `libc++`, despite the fact that this could break some third-party code that assumes that integers are no wider than 64 bits and no wider than `std::intmax_t`. The real problems arise only for small signed bit-precise integers such as `_BitInt(1)` and for huge bit-precise integers such as (possibly `unsigned`) `_BitInt(32'768)`.
7. Making `std::is_integral_v<_BitInt(N)> false` is ineffective when considering that by design, the category of integral types has always been extendable using extended integer types. These are not compiler extensions, but rather an open set of optional types that can even be exposed using aliases such as `std::int128_t`. Even more extremely, the implementation can decide to add a set of extended integer types such as `_ExtInt(N)` with essentially identical behavior to `_BitInt(N)`, and the standard would require `std::is_integral_v<_ExtInt(N)>` to be `true`. In fact, `_ExtInt(N)` is currently an alternative spelling for `_BitInt(N)` in Clang. It would be inconsistent to say it's fine for an

arbitrary open set of implementation-specific types to be integral while saying that this is not fine for `_BitInt(N)`.



8. If `_BitInt(N)` was to receive more standard library support in the future to a point where `std::is_integral_v<_BitInt(N)>` being `false` becomes undesirable from LEWG's perspective, it would be a breaking change to make it `true` later. We can decide to make it whatever we want right now, but this decision is likely irreversible. In the long term, it would make the language design terribly confusing if `_BitInt` had extensive core language and standard library support, but was not considered an integral type. This is the worst possible design outcome.
9. C++ has always reserved the right to extend type sets and continues to do so. For example, the set of floating-point types was extended to include extended floating-point types such as `std::float128_t` and `std::bfloat16_t`, which could also break user expectations regarding floating-point formats. It is also not unlikely that decimal floating-point types will receive a standard alias in the future, such as `std::decimal64_t` to match `_Decimal64` in C. Are we then going to say that decimal floating-point types are not floating-point types?! Note that the implementation can already provide `_Decimal64` as an extended floating-point type right now, so this situation is exactly the same as with providing `_ExtInt(N)` as an extended integer type: it can *technically* be done already without being classified as a compiler extension, and the type traits match the user expectations in that case.

§ 5.2.2. Conclusion

Given the above reasons, I cannot with good conscience support the LEWG decision to make `std::is_integral_v<_BitInt(N)>` `false`. It requires relitigation, especially since much of the rationale above was not discussed in Croydon, such as the existing libc++ behavior.

§ 5.3. `make_signed` and `make_unsigned`

To prevent breaking existing code, the behavior of `make_signed` and `make_unsigned` needs to be made future-proof:

```
make_unsigned_t<char32_t> // previously unsigned int, becomes _BitInt(32) unless
```

The rank of `unsigned int` is greater than the rank of `unsigned _BitInt(32)` (assuming those have the same width; see [\[conv.rank\]](#)). Therefore, `make_unsigned_t` would need to be `unsigned _BitInt(32)`, since it produces ([\[meta.trans.sign\]](#))

”

```
unsigned integer type with smallest rank ([conv.rank]) for which sizeof(T) ==
sizeof(type)
```

Furthermore, the current wording would give the user an implementation-defined type in the following scenario:

```
enum E : _BitInt(32) { };
make_signed_t<E> x; // might be _BitInt(32)
```


`make_signed_t<E>` could be either `_BitInt(32)` or an extended integer type with lower conversion rank than `_BitInt(32)`. However, for simplicity, `make_signed` and `make_unsigned` should always produce a bit-precise integer type when they are fed a bit-precise integer type or an enumeration whose underlying type is a bit-precise integer.

Overall, `make_signed` can be made future-proof with the following set of rules:

- For signed integers and unsigned integers, it does the "obvious thing".
- For types whose underlying type is a bit-precise integer, it behaves like `make_signed_t<underlying_type_t<T>>`. This only affects enumeration types, since integral types like `char32_t` are currently specified not to have a bit-precise underlying type.
- For any other integral type (`char32_t`, other enumerations, etc.), it denotes the smallest **standard or extended** signed integer type that fits that integral type.

`make_unsigned` should behave correspondingly.



NOTE: See § [\[meta.trans.sign\]](#) for wording.

§ 5.4. The problem of representing widths as `int`

A pre-existing and prolific issue in the C++ standard library is the use of `int` to represent properties of integers, such as

- `int std::numeric_limits::digits` or
- `int std::popcount(T)`.

This has never been a practical issue before, it is now theoretically possible that an implementation may want to provide `_BitInt(32'768)` or wider. `int` is only guaranteed to have the range of a 16-bit signed integer, so it may not be able to represent such huge widths.

The easiest solution is to ignore the problem; this is proposed. It would require substantial design changes to `<limits>` to fix the issue. Furthermore, the practical utility of `_BitInt(32'768)` and wider is somewhat questionable, especially on 16-bit architectures (which are typically embedded architectures). On 32-bit architectures and above, `int` is typically 32-bit, so this problem doesn't exist.

§ 5.5. Preventing `ranges::iota_view` ABI break

Due to the current wording in [\[range.iota.view\] paragraph 1](#), adding bit-precise integers or extended integers of greater width than `long long` potentially forces the implementation to redefine `ranges::iota_view::iterator::difference_type`. Changing the type would be an ABI break. This problem is similar to historical issues with `intmax_t`, where adding 128-bit integers would force the implementation to redefine the former type.

To prevent this, the proposal tweaks the wording in § [\[range.iota.view\]](#) so that new extended or bit-precise integers may be added. Dealing with extended integer types extends slightly beyond

the scope of the MVP, but it would be silly to leave the wording in an undesirable state, where adding a 128-bit extended integer still forces an ABI break.



§ 5.6. Preserving integer-class types

Another very similar wording issue to the one in the previous section arises for the so-called "integer-class types" in the standard library, in [\[iterator.concept.winc\] paragraph 3](#). Signed-integer-like types are either signed integral types, or signed-integer-class types. Integer-class types are required to be wider than every integral type of the same signedness, so introducing bit-precise integers such as `_BitInt(128)` means that e.g. Microsoft's `std::_Signed128` is no longer an integer-class type, and may no longer be used in `ranges::iota_view`.

§ 5.7. Bit-precise `size_t`, `ptrdiff_t`

As in C, the proposal allows for `size_t` and `ptrdiff_t` to be bit-precise integers, which is a consequence of `sizeof` and pointer subtraction potentially yielding a bit-precise integer.

We don't need to explicitly disallow this; it is effectively disallowed because the lack of `_BitInt` support in the standard library would result in a dysfunctional implementation if `size_t`, `ptrdiff_t`, or any `size_t` types were bit-precise integers.

§ 5.8. Feature testing

After consulting with some LWG and SG10 experts, I have opted to add only two feature-test macros: one for the core feature, and one for the standard library. While more granular feature-testing could be useful considering that the feature is quite large, there seems to be little enthusiasm for it.

§ 5.9. Using bit-precise integers in `<cmath>` functions

The proposal adds support for using bit-precise integers in all `<cmath>` functions:

```
std::sqrt(0);    // OK, int → call to std::sqrt(double)
std::sqrt(0wb); // OK, _BitInt(1) → call to std::sqrt(double)
```

This is done simply for consistency with C: after some consulting with WG14 members, I am under the impression that C's `<tgmath.h>` functions deliberately support all integer types (including bit-precise integers), not just as the result of defective wording. Consequently, `_BitInt` can be passed both to the type-generic `sqrt` macro as well as to the regular `sqrt(double)` function.

§ 5.10. Note on alias templates for `_BitInt`



NOTE: Up to R3 of this paper, the proposal provided the following alias templates:


```
template <size_t N>
using bit_int = _BitInt(N);
template <size_t N>
using bit_uint = unsigned _BitInt(N);
```



LEWG decided against these templates, so they are no longer part of this paper:

”

POLL: We should provide alias templates for `_BitInt()` (i.e. `std::bit_int` and `std::bit_uint` – possibly under a different name e.g. `std::cbit_int`)

SF	F	N	A	SA
0	3	8	3	5

Outcome: No consensus

§ 6. Implementation experience

`_BitInt`, formerly known as `_ExtInt`, has been a compiler extension in Clang for several years now. The core language changes are essentially standardizing that compiler extension.

When compiling using Clang and libstdc++, one gets virtually the proposed behavior. That is, just the core language feature, with minimal standard library support.



NOTE: There are still minor differences in that scenario because in libstdc++, `std::is_integral_v<_BitInt(N)>` is `false`.

§ 7. Impact on the standard

§ 7.1. Impact on the core language

The core language changes essentially boil down to adding the `_BitInt` type and the *wb integer-suffix*. This obviously comes with various syntax changes, definitions of conversion rank, addition of template argument deduction rules, etc. The vast majority of core language wording which deals with integers is not affected by the existence of bit-precise integers.

§ 7.2. Impact on the standard library

As explained in §5. [Library design](#), various constraints are added throughout the standard library to prevent support for bit-precise integers.

§ 8. Wording

The following changes are relative to [N5032] with the changes from Croydon motions applied.



§ 8.1. Core



DECISION: CWG needs to decide what the “quoted” (prose) spelling of bit-precise integer types should be. The current spelling is e.g. “`unsigned _BitInt` of width N ”, which is fairly similar to other code-heavy spellings like “`unsigned int`”.

However, this is questionable because `_BitInt` is not valid C++ in itself; `_BitInt(N)` is. An alternative would be a pure prose spelling, like “bit-precise unsigned integer of width N ”, which is a bit more verbose.

There is no strong author preference.

§ [lex.icon]

In [lex.icon], change the grammar as follows:



```
integer-suffix:
    unsigned-suffix long-suffixopt
    unsigned-suffix long-long-suffixopt
    unsigned-suffix size-suffixopt
    unsigned-suffix bit-precise-int-suffixopt
    long-suffix unsigned-suffixopt
    long-long-suffix unsigned-suffixopt
    size-suffix unsigned-suffixopt
    bit-precise-int-suffix unsigned-suffixopt

unsigned-suffix: one of
    u U

long-suffix: one of
    l L

long-long-suffix: one of
    ll LL

size-suffix: one of
    z Z

bit-precise-int-suffix: one of
    wb WB
```



DRAFTING NOTE: The name *bit-precise-int-suffix* is identical to the one used in C. See [N3550] §6.4.5.2 Integer literals.

Change table [tab:lex.icon.type] as follows:



<i>integer-suffix</i>	<i>decimal-literal</i>	<i>integer-literal other than decimal-literal</i>
none	“int” “long int” “long long int”	“int” “unsigned int” “long int” “unsigned long int” “long long int” “unsigned long long int”
u or U	“unsigned int” “unsigned long int” “unsigned long long int”	“unsigned int” “unsigned long int” “unsigned long long int”
l or L	“long int” “long long int”	“long int” “unsigned long int” “long long int” “unsigned long long int”
Both u or U and l or L	“unsigned long int” “unsigned long long int”	“unsigned long int” “unsigned long long int”
Both u or U and ll or LL	“unsigned long long int”	“unsigned long long int”
z or Z	the signed integer type corresponding to <u>the type named by</u> <code>std::size_t</code> (<code>[support.types.layout]</code>)	the signed integer type corresponding to <u>the type named by</u> <code>std::size_t</code> <u>the type named by</u> <code>std::size_t</code>
Both u or U and z or Z	<u>the type named by</u> <code>std::size_t</code>	<u>the type named by</u> <code>std::size_t</code>
<u>wb or WB</u>	“ <u>BitInt of width N</u> ”, where N is the lowest integer ≥ 1 so that <u>the value of the literal can be represented by the type</u>	“ <u>BitInt of width N</u> ”, where N is the lowest integer ≥ 1 so that <u>the value of the literal can be represented by the type</u>
<u>Both u or U and wb or WB</u>	“ <u>unsigned BitInt of width N</u> ”, where N is the lowest integer ≥ 1 so that the value of the <u>literal can be represented by the type</u>	“ <u>unsigned BitInt of width N</u> ”, where N is the lowest integer ≥ 1 so that the value of the <u>literal can be represented by the type</u>



DRAFTING NOTE: The existing rows are adjusted for consistency. We usually aim to use the “quoted” spellings of types like “BitInt of width N ” in core wording instead of the *type-id* spellings. Adding a quoted spelling for bit-precise integers would reveal that the previous rows “incorrectly” use *type-ids*.

Change [\[lex.icon\]](#) paragraph 4 as follows:



Except for *integer-literals* containing a *size-suffix* or bit-precise-int-suffix, if the value of an *integer-literal* cannot be represented by any type in its list and an extended integer type

([basic.fundamental]) can represent its value, it may have that extended integer type. [...]



[Note: An *integer-literal* with a `z` or `Z` suffix is ill-formed if it cannot be represented by `std::size_t`. An integer-literal with a `wb` or `WB` suffix is ill-formed if it cannot be represented by any bit-precise integer type because the necessary width is greater than `BITINT_MAXWIDTH` ([climits.syn]). — end note]

§ [basic.fundamental]

Change [basic.fundamental] paragraph 1 as follows:



There are five *standard signed integer types*: “`signed char`”, “`short int`”, “`int`”, “`long int`”, and “`long long int`”. In this list, each type provides at least as much storage as those preceding it in the list. There is also a distinct bit-precise signed integer type “`_BitInt` of width N ” for each $1 \leq N \leq \text{BITINT_MAXWIDTH}$ ([climits.syn]). There may also be implementation-defined *extended signed integer types*. The standard, bit-precise, and extended signed integer types are collectively called *signed integer types*. The range of representable values for a signed integer type is -2^{N-1} to $2^{N-1} - 1$ (inclusive), where N is called the *width* of the type.

[Note: Plain `ints` are intended to have the natural width suggested by the architecture of the execution environment; the other signed integer types are provided to meet special needs. — end note]



DRAFTING NOTE: This change deviates from C at the time of writing; C2y does not yet allow `_BitInt(1)`, but may allow it following [N3699].

Change [basic.fundamental] paragraph 2 as follows:



For each of the standard signed integer types, there exists a corresponding (but different) *standard unsigned integer type*: “`unsigned char`”, “`unsigned short`”, “`unsigned int`”, “`unsigned long int`”, and “`unsigned long long int`”. For each bit-precise signed integer type “`_BitInt` of width N ”, there exists a corresponding bit-precise unsigned integer type “`unsigned _BitInt` of width N ”. ~~Likewise, for~~ For each of the extended signed integer types, there exists a corresponding *extended unsigned integer type*. The standard, bit-precise, and extended unsigned integer types are collectively called *unsigned integer types*. An unsigned integer type has the same width N as the corresponding signed integer type. The range of representable values for the unsigned type is 0 to $2^N - 1$ (inclusive); arithmetic for the unsigned type is performed modulo 2^N .

[Note: Unsigned arithmetic does not overflow. Overflow for signed arithmetic yields undefined behavior ([expr.pre]). — end note]

Change [basic.fundamental] paragraph 5 as follows:



[...] The standard signed integer types and standard unsigned integer types are collectively called the *standard integer types*, ~~and the~~ The bit-precise signed integer types and bit-precise unsigned integer types are collectively called the *bit-precise integer types*. The extended signed integer types and extended unsigned integer types are collectively called the *extended integer types*.

§ [conv.rank]

Change [conv.rank] paragraph 1 as follows:



Every integer type has an *integer conversion rank* defined as follows:

- No two signed integer types other than `char` and `signed char` (if `char` is signed) have the same rank, even if they have the same representation.
- The rank of a signed integer type is greater than the rank of any signed integer type with a smaller width.
- The rank of `long long int` is greater than the rank of `long int`, which is greater than the rank of `int`, which is greater than the rank of `short int`, which is greater than the rank of `signed char`.
- The rank of any unsigned integer type equals the rank of the corresponding signed integer type.
- The rank of any standard integer type is greater than the rank of any bit-precise integer type with the same width and of any extended integer type with the same width.
- The rank of `char` equals the rank of `signed char` and `unsigned char`.
- The rank of `bool` is less than the rank of all standard integer types.
- The ranks of `char8_t`, `char16_t`, `char32_t`, and `wchar_t` equal the ranks of their underlying types ([basic.fundamental]).
- The rank of any extended signed integer type relative to another extended signed integer type with the same width and relative to a bit-precise signed integer type with the same width is implementation-defined, but still subject to the other rules for determining the integer conversion rank.
- For all integer types T1, T2, and T3, if T1 has greater rank than T2 and T2 has greater rank than T3, then T1 has greater rank than T3.

[Note: The integer conversion rank is used in the definition of the integral promotions ([conv.prom]) and the usual arithmetic conversions ([expr.arith.conv]). — end note]

§ [conv.prom]



DRAFTING NOTE: These changes mirror the C semantics described in [N3550] §6.3.2.1 Boolean, characters, and integers.

Change [conv.prom] paragraph 2 as follows:



A prvalue that



- is not a converted bit-field **and**,
- has an integer type other than **a bit-precise integer type**, `bool`, `char8_t`, `char16_t`, `char32_t`, or `wchar_t`, **and**
- whose integer conversion rank (`[conv.rank]`) is less than the rank of `int`

can be converted to a prvalue of type `int` if `int` can represent all the values of the source type; otherwise, the source prvalue can be converted to a prvalue of type `unsigned int`.

Change `[conv.prom]` paragraph 3 as follows:



A prvalue of an unscoped enumeration type whose underlying type is not fixed¹ can be converted to a prvalue of the first of the following types that can represent all the values of the enumeration (`[dcl.enum]`): `int`, `unsigned int`, `long int`, `unsigned long int`, `long long int`, or `unsigned long long int`. If none of the types in that list can represent all the values of the enumeration, a prvalue of an unscoped enumeration type **whose underlying type is not a bit-precise integer type** can be converted to a prvalue of the extended integer type with lowest integer conversion rank (`[conv.rank]`) greater than the rank of `long long` in which all the values of the enumeration can be represented. If there are two such extended types, the signed one is chosen.

1) This promotion rule excludes bit-precise integers because the implementation cannot choose a bit-precise integer type as the underlying type of an enumeration with no fixed underlying type (`[dcl.enum]`).

Change `[conv.prom]` paragraph 4 as follows:



A prvalue of an unscoped enumeration type whose underlying type is fixed (`[dcl.enum]`) can be converted to a prvalue of its underlying type. Moreover, if integral promotion can be applied to its underlying type, a prvalue of an unscoped enumeration type whose underlying type is fixed can also be converted to a prvalue of the promoted underlying type.

[Note: A converted bit-field of enumeration type is treated as any other value of that type for promotion purposes. — end note]

[Note: If the underlying type is a bit-precise integer type, conversion to a prvalue of that type is possible, but integral promotion cannot be applied to the underlying type. — end note]

Change `[conv.prom]` paragraph 5 as follows:



A converted bit-field of integral type **other than a bit-precise integer type** can be converted to a prvalue of type `int` if `int` can represent all the values of the bit-field;

otherwise, it can be converted to `unsigned int` if `unsigned int` can represent all the values of the bit-field.



§ [dcl.type.general]

Change [dcl.type.general] paragraph 2 as follows:



As a general rule, at most one *defining-type-specifier* is allowed in the complete *decl-specifier-seq* of a declaration or in a *defining-type-specifier-seq*, and at most one *type-specifier* is allowed in a *type-specifier-seq*. The only exceptions to this rule are the following:

- `const` can be combined with any type specifier except itself.
- `volatile` can be combined with any type specifier except itself.
- `signed` or `unsigned` can be combined with `char`, `long`, `short`, ~~or~~ `int`, or a bit-precise-int-type-specifier ([dcl.type.simple]).
- `short` or `long` can be combined with `int`.
- `long` can be combined with `double`.
- `long` can be combined with `long`.

§ [dcl.type.simple]

Change [dcl.type.simple] paragraph 1 as follows:



The simple type specifiers are

```
simple-type-specifier:  
    nested-name-specifieropt type-name  
    nested-name-specifier template simple-template-id  
    computed-type-specifier  
    placeholder-type-specifier  
    bit-precise-int-type-specifier  
    nested-name-specifieropt template-name  
  
    char  
    char8_t  
    char16_t  
    char32_t  
    wchar_t  
    bool  
    short  
    int  
    long  
    signed  
    unsigned  
    float
```




```
double
void

type-name:
    class-name
    enum-name
    typedef-name

computed-type-specifier:
    decltype-specifier
    pack-index-specifier
    splice-type-specifier

bit-precise-int-type-specifier:
    BitInt ( constant-expression )
```



DRAFTING NOTE: The name *bit-precise-int-type-specifier* is symmetrical with *bit-precise-int-suffix*.

Change table [tab:dcl.type.simple] as follows:



Specifier(s)	Type
<i>type-name</i>	the type named
<i>simple-template-id</i>	the type as defined in [temp.names]
<i>decltype-specifier</i>	the type as defined in [dcl.type.decltype]
<i>pack-index-specifier</i>	the type as defined in [dcl.type.pack.index]
<i>placeholder-type-specifier</i>	the type as defined in [dcl.spec.auto]
<i>template-name</i>	the type as defined in [dcl.type.class.deduct]
<i>splice-type-specifier</i>	the type as defined in [dcl.type.splice]
<u>unsigned BitInt(N)</u>	<u>“unsigned BitInt of width N”</u>
<u>signed BitInt(N)</u>	<u>“ BitInt of width N”</u>
<u>BitInt(N)</u>	<u>“ BitInt of width N”</u>
char	“char”
unsigned char	“unsigned char”
signed char	“signed char”
char8_t	“char8_t”
char16_t	“char16_t”
char32_t	“char32_t”
bool	“bool”
unsigned	“unsigned int”
unsigned int	“unsigned int”
signed	“int”
signed int	“int”
int	“int”
unsigned short int	“unsigned short int”
unsigned short	“unsigned short int”



<code>unsigned long int</code>	<code>"unsigned long int"</code>
<code>unsigned long</code>	<code>"unsigned long int"</code>
<code>unsigned long long int</code>	<code>"unsigned long long int"</code>
<code>unsigned long long</code>	<code>"unsigned long long int"</code>
<code>signed long int</code>	<code>"long int"</code>
<code>signed long</code>	<code>"long int"</code>
<code>signed long long int</code>	<code>"long long int"</code>
<code>signed long long</code>	<code>"long long int"</code>
<code>long long int</code>	<code>"long long int"</code>
<code>long long</code>	<code>"long long int"</code>
<code>long int</code>	<code>"long int"</code>
<code>long</code>	<code>"long int"</code>
<code>signed short int</code>	<code>"short int"</code>
<code>signed short</code>	<code>"short int"</code>
<code>short int</code>	<code>"short int"</code>
<code>short</code>	<code>"short int"</code>
<code>wchar_t</code>	<code>"wchar_t"</code>
<code>float</code>	<code>"float"</code>
<code>double</code>	<code>"double"</code>
<code>long double</code>	<code>"long double"</code>
<code>void</code>	<code>"void"</code>

Immediately following [\[dcl.type.simple\]](#) paragraph 3, add a new paragraph as follows:



Within a *bit-precise-int-type-specifier*, the *constant-expression* shall be a converted constant expression of type `std::size_t` ([\[expr.const\]](#)). Its value N specifies the width of the bit-precise integer type ([\[basic.fundamental\]](#)). The program is ill-formed unless $1 \leq N \leq \text{BITINT_MAXWIDTH}$ ([\[climits.syn\]](#)).



DRAFTING NOTE: This added paragraph is inspired by [\[dcl.array\]](#) paragraph 1, which similarly specifies the array size to be a converted constant expression of type `std::size_t`.

§ [\[dcl.enum\]](#)



DRAFTING NOTE: The intent is to ban `_BitInt` from *implicitly* being the underlying type of enumerations, matching the proposed restrictions in [\[N3705\]](#). See §4.3. *Underlying type of enumerations*.

Change [\[dcl.enum\]](#) paragraph 5 as follows:



[...] If the underlying type is not fixed, the type of each enumerator prior to the closing brace is determined as follows:

- If an initializer is specified for an enumerator, the *constant-expression* shall be an integral constant expression ([[expr.const](#)]) whose type is not a bit-precise integer type. If the expression has unscoped enumeration type, the enumerator has the underlying type of that enumeration type, otherwise it has the same type as the expression.
- If no initializer is specified for the first enumerator, its type is an unspecified signed ~~integral~~ integer type other than a bit-precise integer type.
- Otherwise, the type of the enumerator is the same as that of the preceding enumerator, unless the incremented value is not representable in that type, in which case the type is an unspecified integral type other than a bit-precise integer type sufficient to contain the incremented value. If no such type exists, the program is ill-formed.

Change [[dcl.enum](#)] paragraph 7 as follows:



For an enumeration whose underlying type is not fixed, the underlying type is an integral type that can represent all the enumerator values defined in the enumeration. If no integral type can represent all the enumerator values, the enumeration is ill-formed. It is implementation-defined which integral type is used as the underlying type, except that

- the underlying type shall not be a bit-precise integer type and
- the underlying type shall not be larger than `int` unless the value of an enumerator cannot fit in an `int` or `unsigned int`.

If the *enumerator-list* is empty, the underlying type is as if the enumeration had a single enumerator with value 0.

§ [[temp.deduct.general](#)]

Add a bullet to [[temp.deduct.general](#)] paragraph 11 as follows:



[*Note*: Type deduction can fail for the following reasons:

- Attempting to instantiate a pack expansion containing multiple packs of differing lengths.
- Attempting to create an array with an element type that is `void`, a function type, or a reference type, or attempting to create an array with a size that is zero or negative.

[*Example*:

```
template <class T> int f(T[5]);
int I = f<int>(0);
int j = f<void>(0);           // invalid array
```

— *end example*]



- Attempting to create a bit-precise integer type of invalid width ([basic.fundamental]).

[Example:

```
template <int N> void f(_BitInt(N));  
f<0>(0); // invalid bit-precise integer
```

— end example]

- [...]

— end note]

§ [temp.deduct.type]

Change [temp.deduct.type] paragraph 2 as follows:



[...] The type of a type parameter is only deduced from an array bound or bit-precise integer width if it is not otherwise deduced.

Change [temp.deduct.type] paragraph 3 as follows:



A given type P can be composed from a number of other types, templates, and constant template argument values:

- A function type includes the types of each of the function parameters, the return type, and its exception specification.
- A pointer-to-member type includes the type of the class object pointed to and the type of the member pointed to.
- A type that is a specialization of a class template (e.g., A<int>) includes the types, templates, and constant template argument values referenced by the template argument list of the specialization.
- An array type includes the array element type and the value of the array bound.
- A bit-precise integer type includes the integer width.

Change [temp.deduct.type] paragraph 5 as follows:



The non-deducted contexts are:

- [...]
- A constant template argument **or**, an array bound, or a bit-precise integer width, in any of which a subexpression references a template parameter.
[Example:

```
template<size_t N> void f(_BitInt(N));
template<size_t N> void g(_BitInt(N + 1));
f(100wb); // OK, N = 8
g(100wb); // error: no argument for ded
— end example]
```

- [...]

Change [\[temp.deduct.type\] paragraph 8](#) as follows:



A type template argument T , a constant template argument i , a template template argument TT denoting a class template or an alias template, or a template template argument VV denoting a variable template or a concept can be deduced if P and A have one of the following forms:

```
CVopt T
T*
T&
T&&
Topt[iopt]
_BitInt(iopt)
Topt(Topt) noexcept(iopt)
Topt Topt::*
TTopt<T>
TTopt<i>
TTopt<TT>
TTopt<VV>
TTopt<>
```

where [...]

Do not change [\[temp.deduct.type\] paragraph 14](#); it is included here for reference.



The type of N in the type $T[N]$ is `std::size_t`.

[Example:

```
template<typename T> struct S;
template<typename T, T n> struct S<int[n]> {
    using Q = T;
};

using V = decltype(sizeof 0);
using V = S<int[42]>::Q; // OK; T was deduced as std::size_t from the typ
— end example]
```

Immediately following [\[temp.deduct.type\] paragraph 14](#), insert a new paragraph:



The type of N in the type `_BitInt(N)` is `std::size_t`.

[Example:

```
template <typename T, T n> void f(_BitInt(n));
```

```
f(0wb); // OK; T was deduced as std::size_t from an argu
```

— end example]



Change [temp.deduct.type] paragraph 20 as follows:



If P has a form that contains `<i>`, and if the type of `i` differs from the type of the corresponding template parameter of the template named by the enclosing *simple-template-id* or *splice-specialization-specifier*, deduction fails. If P has a form that contains `[i] or _BitInt(i)`, and if the type of `i` is not an integral type, deduction fails. If P has a form that includes `noexcept(i)` and the type of `i` is not `bool`, deduction fails.

§ [cpp.predefined]

Add a feature-test macro to the table in [cpp.predefined] as follows:



```
__cpp_bit_int 20XXXXL
```

§ [diff.lex]



DRAFTING NOTE: See §4.5. `_BitInt(1)`.

In [diff.lex], add a new entry:



Affected subclause: [lex.icon]

Change: The type of `0wb` is changed from `_BitInt(2)` to `_BitInt(1)`.

Rationale: It is expected that a future C standard makes the same change, as part of making `_BitInt(1)` a valid type.

Effect on the original feature: Change to semantics of well-defined feature.

Difficulty of converting: Usually, no changes are required because the type of `0wb` is inconsequential.

How widely used: Seldom.

§ 8.2. Library

§ [allocator.requirements.general]

Change [allocator.requirements.general] as follows:



```
typename X::size_type
```

Result: An standard unsigned or extended unsigned integer type that can represent the size of the largest object in the allocation model.

Remarks: Default: `make_unsigned_t<XX::difference_type>`



`typename X::difference_type`

Result: A standard signed or extended signed integer type that can represent the difference between any two pointers in the allocation model.

Remarks: Default: `pointer_traits<XX::pointer>::difference_type`

§ [version.syn]

Add the following feature-test macro to [version.syn]:



```
#define __cpp_lib_bit_int
```

```
20XXXXL
```

§ [support.types.byteops]

Change [support.types.byteops] as follows:



```
template<class IntType>
constexpr byte& operator<=(byte& b, IntType shift) noexcept;
```

Constraints: ~~`is_integral_v<IntType> is true`~~: IntType is an integral type other than a possibly cv-qualified bit-precise integer type.

Effects: Equivalent to: `return b = b << shift;`

```
template<class IntType>
constexpr byte operator<<(byte b, IntType shift) noexcept;
```

Constraints: ~~`is_integral_v<IntType> is true`~~: IntType is an integral type other than a possibly cv-qualified bit-precise integer type.

Effects: Equivalent to: `return static_cast<byte>(static_cast<unsigned int>(b) << shift);`

```
template<class IntType>
constexpr byte& operator>=(byte& b, IntType shift) noexcept;
```

Constraints: ~~`is_integral_v<IntType> is true`~~: IntType is an integral type other than a possibly cv-qualified bit-precise integer type.

Effects: Equivalent to: `return b = b >> shift;`

```
template<class IntType>
constexpr byte operator>>(byte b, IntType shift) noexcept;
```

Constraints: ~~`is_integral_v<IntType> is true`~~: IntType is an integral type other than a possibly cv-qualified bit-precise integer type.

Effects: Equivalent to: `return static_cast<byte>(static_cast<unsigned int>(b) >> shift);`



[...]

```
template<class IntType>
constexpr IntType to_integer(byte b) noexcept;
```

Constraints: ~~`is_integral_v<IntType> is true`~~. IntType is an integral type other than a possibly cv-qualified bit-precise integer type.

Effects: Equivalent to: `return static_cast<IntType>(b);`

§ [cstdint.syn]

Change [cstdint.syn] paragraph 2 as follows:



The header defines all types and macros the same as the C standard library header `<stdint.h>`. None of the aliases name a bit-precise integer type. The types denoted by `intmax_t` and `uintmax_t` are not required to be able to represent all values of bit-precise integer types or of extended integer types wider than “`long long int`” and “`unsigned long long int`”, respectively.

Change [cstdint.syn] paragraph 3 as follows:



All types that use the placeholder *N* are optional when *N* is not 8, 16, 32, or 64. The exact-width types `intN_t` and `uintN_t` for *N* = 8, 16, 32, and 64 are also optional; however, if an implementation defines integer types other than bit-precise integer types with the corresponding width and no padding bits, it defines the corresponding *typedef-names*. Each of the macros listed in this subclause is defined if and only if the implementation defines the corresponding *typedef-name*.
[Note: The macros `INTN_C` and `UINTN_C` correspond to the *typedef-names* `int_leastN_t` and `uint_leastN_t`, respectively. — end note]

§ [climits.syn]

In [climits.syn], add a new line below the definition of `ULLONG_WIDTH`:



```
#define BITINT_MAXWIDTH see below
```

Change the synopsis in [climits.syn] paragraph 1 as follows:



The header `<climits>` defines all macros the same as the C standard library header `limits.h`, ~~except that it does not define the macro `BITINT_MAXWIDTH`.~~

§ [intseq.intseq]

Change [intseq.intseq] as follows:



```
namespace std {
    template<class T, T... I> struct integer_sequence {
        using value_type = T;
        static constexpr size_t size() noexcept { return sizeof...(I); }
    };
}
```

Mandates: T is an integer type other than a possibly cv-qualified bit-precise integer type.

§ [meta.trans.sign]



DRAFTING NOTE: See §5.3. make_signed and make_unsigned.

Change table [tab:meta.trans.sign] as follows:



Template	Comments
<code>template<class T> struct make_signed;</code>	<u>Specializations have an alias member type determined as follows:</u> • If T is a (possibly cv-qualified) signed integer type ([basic.fundamental]) then the member typedef , type denotes T ; . • otherwise <u>Otherwise</u>, if T is a (possibly cv-qualified) <u>an</u> unsigned integer type then , type denotes the corresponding signed integer type , with the same cv-qualifiers as T; . • <u>Otherwise, if T's underlying type U is a bit-precise signed integer type, type denotes U.</u> • <u>Otherwise, if T's underlying type U is a bit-precise unsigned integer type, type denotes the corresponding signed integer type of U.</u> • otherwise <u>Otherwise, if T is cv-unqualified</u>, type denotes the <u>standard or extended</u> signed integer type with smallest rank ([conv.rank]) for which <code>sizeof(T) == equals sizeof(type)</code> , with the same cv-qualifiers as T. • <u>Otherwise, T is a cv-qualified type. type denotes the type determined by applying the rules above to remove_cv_t<T>, with the same cv-qualifiers as T.</u> Mandates: T is an integral or enumeration type other than cv bool.

```
template<class T>
struct make_unsigned;
```

Specializations have an alias member type determined as follows:



- If T is a ~~(possibly cv-qualified)~~ unsigned integer type ([basic.fundamental]) ~~then the member typedef~~, type denotes T ~~;~~.
- ~~otherwise~~ Otherwise, if T is a ~~(possibly cv-qualified)~~ signed integer type ~~then~~, type denotes the corresponding unsigned integer type ~~, with the same cv-qualifiers as T;~~.
- Otherwise, if T's underlying type U is a bit-precise unsigned integer type, type denotes U.
- Otherwise, if T's underlying type U is a bit-precise signed integer type, type denotes the corresponding unsigned integer type of U.
- ~~otherwise~~ Otherwise, if T is cv-unqualified, type denotes the standard or extended unsigned integer type with smallest rank ([conv.rank]) for which `sizeof(T) == equals sizeof(type)` ~~, with the same cv-qualifiers as T.~~
- Otherwise, T is a cv-qualified type. type denotes the type determined by applying the rules above to `remove_cv_t<T>`, with the same cv-qualifiers as T.

Mandates: T is an integral or enumeration type other than cv `bool`.

§ [utility.intcmp]

Change [utility.intcmp] as follows:



```
template<class T, class U>
constexpr bool cmp_equal(T t, U u) noexcept;
```

Mandates: Each of T and U is a ~~signed or unsigned~~ standard or extended integer type ([basic.fundamental]).

Effects: [...]

```
template<class T, class U>
constexpr bool cmp_less(T t, U u) noexcept;
```

Mandates: Each of T and U is a ~~signed or unsigned~~ standard or extended integer type ([basic.fundamental]).

Effects: [...]

```
template<class R, class T>
constexpr bool in_range(T t) noexcept;
```



Mandates: Each of T and R is a ~~signed or unsigned~~ standard or extended integer type ([basic.fundamental]).

Effects: [...]

§ [bit.byteswap]

Change [bit.byteswap] as follows:



```
template<class T>
constexpr T byteswap(T value) noexcept;
```

Constraints: T ~~models integral~~ is an integral type other than a possibly cv-qualified bit-precise integer type.

[...]

§ [bit]

Change the *Constraints* element attached to each of the function templates `has_single_bit`, `bit_ceil`, `bit_floor`, `bit_width`, `rotr`, `rotl`, `countl_zero`, `countl_one`, `countr_zero`, `countr_one`, and `popcount` as follows:



Constraints: T is an unsigned integer type other than a bit-precise integer type ([basic.fundamental]).

§ [stdbit.h.syn]



DRAFTING NOTE: The following *Mandates* element applies to the function template in various overload sets such as:

```
unsigned int stdc_leading_zeros_uc(unsigned char value);
unsigned int stdc_leading_zeros_us(unsigned short value);
unsigned int stdc_leading_zeros_ui(unsigned int value);
unsigned int stdc_leading_zeros_ul(unsigned long int value);
unsigned int stdc_leading_zeros_ull(unsigned long long int value);
template<class T> see below stdc_leading_zeros(T value);
```

Change [stdbit.h.syn] paragraph 2 as follows:



Mandates: T is ~~an unsigned integer type~~

- a standard unsigned integer type,
- an extended unsigned integer type, or

- a bit-precise unsigned integer type whose width matches a standard or extended integer type.



§ [container.reqmts]

Change [container.reqmts] as follows:



```
typename X::difference_type
```

Result: A signed integer type other than a possibly cv-qualified bit-precise integer type, identical to the difference type of `X::iterator` and `X::const_iterator`.

```
typename X::size_type
```

Result: An unsigned integer type other than a possibly cv-qualified bit-precise integer type, that can represent any non-negative value of `X::difference_type`.

§ [mdspan.extents.overview]

Change [mdspan.extents.overview] as follows:



```
namespace std {  
    template<class IndexType, size_t... Extents>  
    class extents {  
        [...]  
    };  
  
    [...]  
}
```

Mandates:

- IndexType is a ~~signed or unsigned~~ standard or extended integer type, and
- each element of Extents is either equal to `dynamic_extent`, or is representable as a value of type IndexType.

§ [mdspan.sub.overview]

Change [mdspan.sub.overview] paragraph 2, [mdspan.sub.overview] paragraph 3, and [mdspan.sub.overview] paragraph 4 as follows:



Given a ~~signed or unsigned~~ standard or extended integer type IndexType [...]

§ [mdspan.sub.range.slices]

Change [mdspan.sub.range.slices] as follows:



[...]

```

namespace std {
    template<class OffsetType, class ExtentType, class StrideType>
    struct extent_slice {
        using offset_type = OffsetType;
        using extent_type = ExtentType;
        using stride_type = StrideType;

        [[no_unique_address]] offset_type offset{};
        [[no_unique_address]] extent_type extent{};
        [[no_unique_address]] stride_type stride{};
    };

    template<class FirstType, class LastType, class StrideType = constant_w
    struct range_slice {
        [[no_unique_address]] FirstType first{};
        [[no_unique_address]] LastType last{};
        [[no_unique_address]] StrideType stride{};
    };
}

```

[...]

Mandates: OffsetType, ExtentType, FirstType, LastType, and StrideType are ~~signed or unsigned~~ standard or extended integer types, or model *integral-constant-like*.

[Note: [...] — end note]

§ [iterator.concept.winc]



DRAFTING NOTE: See §5.6. Preserving integer-class types.

Change [iterator.concept.winc] as follows:



[...] The width of an integer-class type is greater than that of every ~~integral~~ standard integer type of the same signedness.

§ [iterator.iterators]

Change [iterator.iterators] paragraph 2 as follows:



A type X meets the *Cpp17Iterator* requirements if

- [...]
- `iterator_traits<X>::difference_type` is a signed integer type other than a bit-precise integer type or `void`, and
- [...]

§ [common.iter.types]



Change [common.iter.types] paragraph 1 as follows:



The nested *typedef-name* `iterator_category` of the specialization of `iterator_traits` for `common_iterator<I, S>` is declared if and only if `iter_difference_t<I>` is an integral type other than a possibly cv-qualified bit-precise integer type.

§ [range.iota.view]



DRAFTING NOTE: See §5.5. Preventing `ranges::iota_view` ABI break.

Change [range.iota.view] paragraph 1 as follows:



Let $IOTA-DIFF-T(W)$ be defined as follows:

- If W is not an integral type, or if it is an integral type and `sizeof(iter_difference_t<W>)` is greater than `sizeof(W)`, then $IOTA-DIFF-T(W)$ denotes `iter_difference_t<W>`.
- Otherwise, $IOTA-DIFF-T(W)$ is a standard signed integer type of width greater than the width of W if such a type exists.
- Otherwise, $IOTA-DIFF-T(W)$ is an unspecified signed-integer-like ([iterator.concept.winc]) type of width not less than the width of W .

§ [alg.foreach]

Change [alg.foreach] `for_each_n` as follows:



```
template<class InputIterator, class Size, class Function>
constexpr InputIterator for_each_n(InputIterator first, Size n, Function
```

Mandates: The type `Size` is convertible to an integral type other than a bit-precise integer type ([conv.integral], [class.conv]).

[...]

```
template<class ExecutionPolicy, class ForwardIterator, class Size, class F>
ForwardIterator for_each_n(ExecutionPolicy&& exec, ForwardIterator first,
                          Function f);
```

Mandates: The type `Size` is convertible to an integral type other than a bit-precise integer type ([conv.integral], [class.conv]).

[...]



DRAFTING NOTE: Implementing this requirement for bit-precise integer types is generally impossible, barring compiler magic. The libc++ implementation is done by calling an overload in the set:

```
int __convert_to_integral(int __val) { return __val; }
unsigned __convert_to_integral(unsigned __val) { return __val; }
```

It is not reasonable to expect millions of additional overloads, and a template that can handle bit-precise integers in bulk could not interoperate with user-defined conversion function templates.

§ [alg.search]

Change [alg.search] paragraph 5 as follows:



Mandates: The type `Size` is convertible to an integral type other than a bit-precise integer type ([conv.integral], [class.conv]).

§ [alg.copy]

Change [alg.copy] paragraph 15 as follows:



Mandates: The type `Size` is convertible to an integral type other than a bit-precise integer type ([conv.integral], [class.conv]).

§ [alg.fill]

Change [alg.fill] paragraph 2 as follows:



Mandates: The expression value is writable ([iterator.requirements.general]) to the output iterator. The type `Size` is convertible to an integral type other than a bit-precise integer type ([conv.integral], [class.conv]).

§ [alg.generate]

Change [alg.generate] paragraph 2 as follows:



Mandates: `Size` is convertible to an integral type other than a bit-precise integer type ([conv.integral], [class.conv]).

§ [numeric.ops.gcd]

Change [numeric.ops.gcd] as follows:



```
template<class M, class N>
constexpr common_type_t<M, N> gcd(M m, N n);
```

Mandates: M and N both are integer types other than cv `bool` or a possibly cv-qualified bit-precise integer type.



[...]

§ [numeric.ops.lcm]

Change [numeric.ops.lcm] as follows:



```
template<class M, class N>
constexpr common_type_t<M, N> lcm(M m, N n);
```

Mandates: M and N both are integer types other than cv `bool` or a possibly cv-qualified bit-precise integer type.

[...]

§ [numeric.ops.midpoint]

Change [numeric.ops.midpoint] as follows:



```
template<class T>
constexpr T midpoint(T a, T b) noexcept;
```

Constraints: T is an arithmetic type other than cv `bool` or a possibly cv-qualified bit-precise integer type.

[...]

§ [numeric.sat.func]

Change the *Constraints* element underneath all function templates in [numeric.sat.func] as follows:



Constraints: T is a ~~signed or unsigned~~ standard or extended integer type ([basic.fundamental]).

§ [numeric.sat.cast]

Change [numeric.sat.cast] as follows:



```
template<class R, class T>
constexpr R saturating_cast(T x) noexcept;
```

Constraints: R and T are ~~signed or unsigned standard or extended~~ integer types ([basic.fundamental]).



§ [charconv.syn]

Change [charconv.syn] paragraph 1 as follows:



When a function is specified with a type placeholder of *integer-type*, the implementation provides overloads for `char` and all ~~cv-unqualified-signed-and-unsigned integer types~~ standard and extended integer types in lieu of *integer-type*. When a function is specified with a type placeholder of *floating-point-type*, the implementation provides overloads for all cv-unqualified floating-point types ([basic.fundamental]) in lieu of *floating-point-type*.

§ [format.formatter.spec]

Change [format.formatter.spec] paragraph 2, bullet 3 as follows:



For each `charT`, for each `IntegerT` that is either a ~~signed-or-unsigned~~ standard or extended integer type or `bool`, a constexpr-enabled specialization

```
template<> struct formatter<IntegerT, charT>;
```

§ [cmplx.over]

Change [cmplx.over] paragraph 2 as follows:



The additional constexpr overloads are sufficient to ensure:

- If the argument has a floating-point type `T`, then it is effectively cast to `complex<T>`.
- Otherwise, if the argument has integral type other than a possibly cv-qualified bit-precise integer type, then it is effectively cast to `complex<double>`.

Change [cmplx.over] paragraph 3 as follows:



Function template `pow` has additional constexpr overloads sufficient to ensure, for a call with one argument of type `complex<T1>` and the other argument of type `T2` or `complex<T2>`, both arguments are effectively cast to `complex<common_type_t<T1, T3>>`, where `T3` is `double` if `T2` is an integer type and `T2` otherwise. If `common_type_t<T1, T3>` is not well-formed or if `T2` is a possibly cv-qualified bit-precise integer type, then the program is ill-formed.

§ [rand.req.seedseq]

In [tab:rand.req.seedseq], change the “Pre-post-condition” column corresponding to `S::result_type` as follows:



T is an unsigned integer type other than a bit-precise integer type of at least 32 bits.



§ [rand.req.urng]

Change [rand.req.urng] paragraph 3 as follows:



A class G meets the *uniform random bit generator* requirements if

- G models `uniform_random_bit_generator`,
- `invoke_result_t<G&>` is an unsigned integer type other than a bit-precise integer type ([basic.fundamental]), and
- G provides a nested *typedef-name* `result_type` that denotes the same type as `invoke_result_t<G&>`.



DRAFTING NOTE: The addition of bullets is part of the change.

§ [rand.util.seedseq]

Change [rand.util.seedseq] as follows:



[...]

```
template<class T>
seed_seq(initializer_list<T> il);
```

Constraints: T is an integer type other than a possibly cv-qualified bit-precise integer type.

Effects: Same as `seed_seq(il.begin(), il.end())`.

```
template<class InputIterator>
seed_seq(InputIterator begin, InputIterator end);
```

Mandates: `iterator_traits<InputIterator>::value_type` is an integer type other than a possibly cv-qualified bit-precise integer type.

[...]

```
template<class RandomAccessIterator>
void generate(RandomAccessIterator begin, RandomAccessIterator end);
```

Mandates: `iterator_traits<RandomAccessIterator>::value_type` is an unsigned integer type other than a possibly cv-qualified bit-precise integer type, capable of accommodating 32-bit quantities.

[...]

§ [cmath.syn]



DRAFTING NOTE: [\[cmath.syn\]](#) paragraph 3 is deliberately not changed, meaning that `bit_int` may be passed to e.g. `sqrt`. See §5.9. Using bit-precise integers in `<cmath>` functions.

§ [simd.expos]

Change [\[simd.expos\]](#) as follows:



```
template<class T>
    concept bit-precise-integer = see below; // exposition only

template<class V>
    concept simd-integral = // exposition only
        simd-vec-type<V> && integral<typename V::value_type> && !bit-precise-
```

§ [simd.expos.defn]

Immediately following the declaration of *deduced-vec-t*, add the following declaration:



```
template<class T>
    concept bit-precise-integer = see below; // exposition only

    The concept bit-precise-integer is satisfied and modeled if and only if T is a
    bit-precise integer type (\[basic.fundamental\]).
```

§ [simd.mask.overview]

Change [\[simd.mask.overview\]](#) as follows:



```
[...]

template<unsigned_integral T>
    requires (!same_as<T, value_type> && !bit-precise-integer<remove_cv_t<T>
    constexpr explicit basic_mask(T val) noexcept;

[...]
```

§ [simd.mask.ctor]

Change [\[simd.mask.ctor\]](#) as follows:



```
[...]

template<unsigned_integral T>
    requires (!same_as<T, value_type> && !bit-precise-integer<remove_cv_t<T>
    constexpr explicit basic_mask(T val) noexcept;
```



§ [numerics.c.ckdint]

Change [numerics.c.ckdint] as follows:



```
template<class type1, class type2, class type3>
    bool ckd_add(type1* result, type2 a, type3 b);
template<class type1, class type2, class type3>
    bool ckd_sub(type1* result, type2 a, type3 b);
template<class type1, class type2, class type3>
    bool ckd_mul(type1* result, type2 a, type3 b);
```

Mandates: Each of the types type1, type2, and type3 is a ~~cv-unqualified signed or unsigned~~ standard or extended integer type.

Remarks: Each function template has the same semantics as the corresponding type-generic macro with the same name specified in ISO/IEC 9899:2024, 7.20.



DRAFTING NOTE: This matches the restrictions in [N3550], 7.20 "Checked Integer Arithmetic". "cv-unqualified" is struck because it is redundant.

§ [time.duration.general]

Change [time.duration.general] paragraph 2 as follows:



Rep shall be an arithmetic type other than a bit-precise integer type or a class emulating an arithmetic type. If a specialization of duration is instantiated with a cv-qualified type or a specialization of duration as the argument for the template parameter Rep, the program is ill-formed.

§ [stream.types]

Change [stream.types] as follows:



```
using streamoff = implementation-defined;
```

The type streamoff is a synonym for ~~one of the signed basic integral types~~ a standard signed integer type of sufficient size to represent the maximum possible file size for the operating system.

```
using streamsize = implementation-defined;
```

The type streamsize is a synonym for ~~one of the signed basic integral types~~ a standard signed integer type. It is used to represent the number of characters transferred in an I/O operation, or the size of I/O buffers.



Change [atomics.ref.int] paragraph 1 as follows:



There are specializations of the `atomic_ref` class template for all integral types **except for possibly cv-qualified bit-precise integer types and** except **for** `cv bool`. [...]

§ 9. Acknowledgements

I thank Jens Maurer and Christof Meerwald for reviewing and correcting the proposal's wording.

I thank Erich Keane and other LLVM contributors for implementing most of the proposed core changes in Clang's C++ frontend, giving this paper years' worth of implementation experience in a major compiler without any effort by the author.

I thank Erich Keane, Jiang An, Bill Seymour, Howard Hinnant, JeanHeyd Meneide, Lénárd Szolnoki, Brian Bi, Peter Dimov, Aaron Ballman, Pete Becker, Jens Maurer, Matthias Kretz, Jonathan Wakely, Jeff Garland, Ville Voutilainen, Peter Dimov, Luigi Ghiron, and *many* others for providing early feedback on this paper, prior papers such as [P3639R0], and the discussion surrounding bit-precise integers as a whole. The paper would not be where it is today without *hundreds* of messages worth of valuable feedback.

§ 10. References

[N1692] *M.J. Kronenburg*. A Proposal to add the Infinite Precision Integer to the C++ Standard Library (2004-07-01)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2004/n1692.pdf>

[N1744] *Michiel Salters*. Big Integer Library Proposal for C++0x (2005-01-13)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2005/n1744.pdf>

[N4038] *Pete Becker*. Proposal for Unbounded-Precision Integer Types (2014-05-23)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4038.html>

[N5032] *Thomas Köppe*. Working Draft, Programming Languages — C++ (2025-12-15)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5032.pdf>

[P3140R0] *Jan Schultke*. `std::int_least128_t` (2025-02-11)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3140r0.html>

[P3161R4] *Tiago Freire*. Unified integer overflow arithmetic (2025-03-24)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3161r4.html>

[P3639R0] *Jan Schultke*. The `_BitInt` Debate (2025-02-20)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3639r0.html>

[P3312R1] *Bengt Gustafsson*. Overload Set Types (2025-04-16)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3312r1.pdf>

[N2763] Aaron Ballman, Melanie Blower, Tommy Hoffner, Erich Keane. Adding a Fundamental Type for N-bit integers (2021-06-21)



<https://open-std.org/JTC1/SC22/WG14/www/docs/n2763.pdf>

[N2775] Aaron Ballman, Melanie Blower. Literal suffixes for bit-precise integers (2021-07-13)

<https://open-std.org/JTC1/SC22/WG14/www/docs/n2775.pdf>

[N3550] JeanHeyd Meneide. ISO/IEC 9899:202y (en) — N3550 working draft (2025-05-04)

<https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3550.pdf>

[N3699] Robert C. Seacord. Integer Sets, v3 (2025-09-02)

<https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3699.pdf>

[N3705] Phillip Klaus Krause. bit-precise enum (2025-09-05)

<https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3705.htm>

[N3747] Robert C. Seacord. Integer Sets, v5 (2025-12-02)

<https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3747.pdf>